UNION - DISCIPLINE - TRAVAIL

Année académique : 2025-2026

MEMOIRE DE FIN DE CYCLE POUR L'OBTENTION DU DIPLOME D'INGENIEUR EN SCIENCE INFORMATIQUE

Conception et implémentation d'une plateforme web de gestion des demandes documentaires et de génération automatisée de documents numériques: CAS DE SYNELIA

Présenté par :

TARNAGDA ISMAEL JEAN-WILFRIED

Encadrant académique :

Dr. AYIKPA KACOU DIEU JEAN

Maitre de stage :

Mr. KOUADIO INNOCENT

DEDICACE

Je dédie ce document à celui-là qui m’a fortifié durant toutes ces années, lui qui ne m’a, malgré tous mes péchés jamais renié, bien au contraire il ne cesse pas de nous protéger. Je veux nommer l’Eternel Dieu tout puissant.

REMERCIEMENTS

Nous voudrions remercier l'école ENSIT (Ecole Nouvelle Supérieur d'Ingénieurs et de Technologie)

et également toutes les personnes qui ont eu à nous aider à la réalisation de ce document notamment :

- L'ensemble des enseignants de l'ENSIT pour les connaissances qu'ils nous ont permis d'acquérir afin de nous guider vers l'excellence.

- Mr Jorge VILANOVA directeur général de SYNELIA qui nous a permis d'entrer dans le milieu professionnel et de nous frotter aux réalités du milieu afin de nous y préparer

- Mr. Innocent KOUADIO mon maitres de stage pour l'encadrement et le conseil reçu

- Tout le personnel administratif de SYNELIA Cote d'Ivoire pour leur accueil qui a permis notre insertion dans leur entreprise

AVANT-PROPOS

Ce document a été rédigé pendant un stage à SYNELIA en vue de présenter notre projet de conception et développement d'une plateforme web sécurisée pour la gestion automatisée des demandes de documents et la génération multi-formats (PDF, Word, Excel), comptant comme projet de fin de cycle pour valider notre Diplôme d'Ingénieur en Sciences Informatiques à l'ENSIT (Ecole Nouvelle Supérieure d'Ingénieurs et de Technologie).

Ce projet vise à digitaliser le système de gestion des demandes de documents chez SYNELIA afin d'automatiser les processus de soumission, validation et génération de documents dans l'entreprise.

Ce document a donc pour objectif de montrer notre solution trouvée à travers les TIC pour résoudre ce problème.

Cette solution permettrait d'améliorer le processus de gestion des demandes de documents chez SYNELIA. L'usage d'outils de programmation et de certaines technologies, telles que Java avec Spring Boot pour le backend et Angular pour le frontend, n'a pas été simple car il s'agissait pour la plupart de nouvelles découvertes ne les ayant jamais utilisées auparavant. Certaines fonctionnalités qui étaient prévues d'être implémentées dans l'application n'ont finalement pas pu être présentes faute au manque de temps eu pour maîtriser et utiliser les technologies nécessaires pour la mise en place de ces fonctions.

TABLES DES ILLUSTRATIONS.

Tableau 1 : Analyse du système de gestion des demandes de documents actuel chez Synelia…..….. 10

Tableau 2: Tableau comparatif des technologies backend. …….....18

Tableau 3: Tableau comparatif des technologies frontend. …….....25

Tableau 4: Tableau comparatif des types de modélisation. …….....31

Tableau 5: Dictionnaire Des données du système.……...…….....40

Figure 1: Schéma de l'organisation de l'entreprise SYNELIA (Côte d'Ivoire).....7

Figure 2 : Schéma de la Structure d'Accès Théorique (SAT)).....44

Figure 3 : Modèle Conceptuel de Données.....48

Figure 4 : Schéma du Modèle Physique des Données (MPD) 52

Figure 5 : MCT 1 : Soumission d'une demande) 54

Figure 6 : MCT 2 : Instruction de la demande (Workflow) 55

Figure 7 : MCT 3 : Production du document.....57

Figure 8 : MCT 4 : Traçabilité et Notification.....58

Figure 9 : Schéma de l'architecture globale du projet.....69

Figure 10 : Architecture Backend.....71

Figure 11 : Architecture Frontend.....73

Figure 12 : Test de l'authentification via Postman.....85

Figure 13.1 : Page d'accueil de l'application…….....83

Figure 13.2 : Page de connexion de l'application.....84

Figure 13.3 : Page de sélection du type de document.....85

Figure 13.4 : Soumission d'une demande documentaire.....86

Figure 13.5 : Validation d'une demande documentaire par Admin.....87

TABLES DES MATIERES

DEDICACE 3

REMERCIEMENTS 4

AVANT-PROPOS 5

TABLES DES ILLUSTRATIONS 6

INTRODUCTION 1

Chapitre 1 : PRESENTATION DE L'ENTREPRISE 4

1. Historique de Synelia 4

2. Activités et domaines d'expertise 4

3. Présentation de Synelia en Côte d'Ivoire 5

II- Missions et objectifs de l'entreprise 5

III- Schéma de l'organisation de l'entreprise Synelia 7

Chapitre 2 : PRESENTATION DU PROJET 9

I- Etude de l'existant 9

1- Présentation de l'existant 9

2- Critique de l'existant 9

3- Solution apportée 10

II- Cahier de charge 10

1- Présentation et contexte du sujet 10

2- Objectifs 11

3- Besoins de l'entreprise 12

3-1. Besoins fonctionnels 12

3-2. Besoins non fonctionnels 13

Chapitre 4 : ÉTUDE ET CHOIX DES SOLUTIONS TECHNIQUES 15

I- Étude comparative des technologies Backend 15

1- Présentation détaillée des technologies 15

2- Critères de comparaison 16

3- Tableau comparatif 17

4- Analyse approfondie des forces et faiblesses 18

5- Conclusion de l'étude comparative 20

1- Présentation détaillée des technologies 22

2- Critères de comparaison 23

3- Tableau comparatif 24

4- Analyse approfondie des forces et faiblesses 25

5- Conclusion de l'étude comparative 27
1- Présentation des méthodes 28
2- Critères de comparaison 29
3- Tableau comparatif 30
4- Analyse approfondie des forces et faiblesses 31
5- Conclusion de l'étude comparative 32
Chapitre 5 : CONCEPTION DU SYSTÈME D'INFORMATION 36
II- Règles de gestion 36
III- Dictionnaire des données 38
IV- Dépendances fonctionnelles 41
VI- Modélisation des données 45
2- Modèle Logique des Données Relationnelles (MLDR) 49
3- Modèle Physique des Données (MPD) 51
Chapitre 6 : REALISATION DE L'APPLICATION 66
I- Environnement et outils de développement 66
1- Environnement matériel et système 66
2- Outils de développement 67
II- Architecture technique de l'application 67
1- Architecture générale 67
2- Architecture backend 69
3- Architecture frontend 71
III- Implémentation des fonctionnalités principales 74
1- Authentification et gestion des utilisateurs 74
2- Gestion des demandes documentaires 76
3- Soumission et workflow de validation 77
4- Gestion des pièces jointes 77
5- Workflow de validation 78
6- Notifications et traçabilité 78
1- Tests et validation de la solution 79
1- Tests fonctionnels 79
2- Tests de sécurité 80
3- Validation globale 80
Chapitre 7 : RESULTAT ET DISCUSSION 82
I- Présentation de l'application réalisée 82
1- Vue générale de l'application 82
2- Pages principales de l'application 83

II- Discussion des résultats obtenus 87
III- Analyse des performances et de la sécurité 88
1- Performances 88
2- Sécurité 88
IV- Analyse des performances et de la sécurité 89
V- Limites et perspectives d'évolution 89
CONCLUSION 91
BIBLIOGRAPHIE 93
WEBOGRAPHIE 94

INTRODUCTION

Une demande de document désigne une requête formulée par un utilisateur (employé, client ou partenaire) pour obtenir un document officiel ou administratif, tel qu'une attestation, un certificat, un rapport ou un contrat. Dans une entreprise, la gestion des demandes de documents représente l'ensemble des processus impliquant la réception, le traitement, la validation et la délivrance de ces documents, souvent sous formats numériques (PDF, Word, Excel). La gestion documentaire joue un rôle essentiel de réservoir tampon entre les flux d'entrées (soumissions de demandes) et de sorties (génération et livraison des documents), permettant à l'entreprise d'assurer flexibilité, traçabilité et conformité réglementaire dans son fonctionnement quotidien.

Synelia est une entreprise spécialisée dans la transformation numérique, leader en Côte d'Ivoire et en Afrique de l'Ouest depuis 2012 (anciennement Smile CI). Elle propose des solutions intégrées en conseil IT, intégration de systèmes, distribution de produits informatiques (partenaires comme Microsoft, HP, Oracle) et accompagnement dans la digitalisation des processus métiers. Dans ce cadre, Synelia traite un volume important de demandes de documents internes et externes, et rencontre des défis liés à leur gestion manuelle ou semi-automatisée.

Le problème de gestion des demandes de documents est courant dans de nombreuses entreprises informatiques : délais prolongés de traitement, erreurs humaines, pertes de données, manque de traçabilité et difficultés à générer des documents sécurisés en multi-formats. Une mauvaise gestion entraîne des risques non négligeables, tels que des non-conformités réglementaires (RGPD ou lois locales sur les données), une perte de productivité et une insatisfaction des utilisateurs. Pour éviter ces risques, il est essentiel de proposer une solution numérique adaptée.

Dans le cadre de notre stage au sein de Synelia, et dans le but de résoudre ce problème, nous travaillons sur le thème suivant : **« Conception et Développement d'une Plateforme Web Sécurisée pour la Gestion Automatisée des Demandes de Documents et la Génération Multi-Formats (PDF, Word, Excel) : Cas de Synelia »**.

L'étude de ce thème vise à automatiser le système de gestion des demandes de documents chez Synelia, le rendant plus flexible, sécurisé et efficace, afin d'assurer un meilleur contrôle sur les flux documentaires et une génération rapide en formats numériques.

Comment automatiser le traitement des demandes de documents ? Comment garantir une génération sécurisée et multi-formats tout en améliorant la traçabilité et les performances ? Telles sont les questions centrales qui guident notre réflexion.

Pour y répondre, nous commencerons par présenter des généralités (entreprise, projet et définitions), puis nous procéderons à l'étude technique (conception et outils), et enfin nous discuterons des résultats (réalisation, tests et perspectives).

PARTIE 1 : GENERALITE

Chapitre 1 : PRESENTATION DE L'ENTREPRISE

I- Présentation générale de l'entreprise

Pour mieux cerner Synelia et les contours de notre sujet, il est indispensable de nous pencher quelques instants sur la présentation de cette entreprise. Cette présentation couvre l'historique, les activités principales, les dimensions de l'entreprise et son organisation.

1. Historique de Synelia

Synelia, implantée en Côte d'Ivoire depuis 2012 sous le nom initial de Smile CI (Smile Côte d'Ivoire), était à ses débuts une filiale franchisée du groupe européen Smile, leader en solutions open source. Bénéficiant de l'expertise et du savoir-faire du groupe européen, Smile CI s'est rapidement imposée comme un acteur clé dans le secteur des technologies de l'information et de la communication (TIC) en Afrique de l'Ouest.

Poussée par son succès, l'entreprise a étendu ses activités à d'autres pays africains, notamment le Liberia et le Burkina Faso. En 2022-2023, dans une démarche d'émancipation et d'ancrage local plus fort, Smile Group Afrique a opéré un rebranding pour devenir **Synelia Group Afrique**, avec un transfert de siège à Abidjan-Cocody Angré. Ce changement reflète l'ambition de l'entreprise de devenir un leader indépendant et pleinement africain dans la transformation numérique.

2. Activités et domaines d'expertise

Synelia est une entreprise spécialisée dans la **transformation numérique**, le conseil IT, l'intégration de systèmes et la distribution de solutions informatiques. Elle accompagne les organisations (entreprises privées, institutions publiques et partenaires internationaux) dans leurs projets digitaux, de la conception à la maintenance.

Ses principaux domaines d'activité incluent :

- Conseil et intégration de solutions open source et propriétaires (partenariats avec Microsoft, HP, Oracle, WALLIX, etc.).

- Cybersécurité : protection des infrastructures numériques, données sensibles et systèmes d'information.
- Infrastructures IT : conception, mise en œuvre et maintenance de data centers, câblage structuré, cloud et réseaux.
- Formation et développement des talents : via des programmes comme Open Digitale Academy, pour renforcer l'employabilité locale.
- Intelligence données, DevOps, IA et applications métier sur mesure.

Synelia se positionne comme un partenaire de confiance pour une transformation numérique agile, sécurisée et adaptée aux contextes africains, avec un focus sur l'innovation locale et l'impact positif sur l'écosystème technologique.

3. Présentation de Synelia en Côte d'Ivoire

Basée à Abidjan (Cocody Angré), Synelia emploie entre 50 et 200 collaborateurs et ambitionne de devenir le leader de la distribution et de la revente de produits informatiques en Côte d'Ivoire. Elle contribue activement à la digitalisation des entreprises ivoiriennes et régionales, en investissant dans la formation des talents locaux pour garantir une employabilité durable.

Synelia accompagne ses clients de bout en bout pour relever les défis de la transformation numérique et les aider à saisir de nouvelles opportunités pour une croissance pérenne et un avantage concurrentiel sur le marché. Grâce à la synergie de ses entités hautement qualifiées et de technologies avancées, Synelia, améliore l'efficience de votre structure en alliant une expertise éprouvée, des technologies de pointe et un esprit agile pour vous guider avec assurance

II- Missions et objectifs de l'entreprise

Synelia propose une capacité d'intervention régionale forte, principalement en Afrique de l'Ouest, avec un ancrage local en Côte d'Ivoire. L'entreprise fournit des solutions de conseil, d'intégration de systèmes, de cybersécurité et d'infrastructures IT qui apportent une valeur ajoutée aux processus numériques de ses clients. Synelia développe des services et solutions informatiques innovants, moteurs essentiels pour l'avenir des organisations africaines.

Elle apporte à ses clients une forte valeur ajoutée et leur assure un avantage concurrentiel significatif grâce à des solutions sur mesure, une maîtrise technique et un esprit collaboratif. Synelia continue à développer ses expertises par la création d'alliances et de partenariats stratégiques (avec Microsoft, HP, Oracle, WALLIX, etc.). Cette approche permet un partage des risques et des bénéfices, tout en anticipant ensemble les besoins du marché africain en pleine expansion numérique. Travailler ensemble à la réalisation des objectifs des clients, anticiper et répondre à leurs attentes constituent le cœur de la philosophie de Synelia.

L'entreprise s'engage à mettre en œuvre et à maintenir un service d'excellence, en poursuivant une démarche de développement continu dans ses relations clients. En tant qu'acteur africain indépendant, Synelia ancre sa **raison d'être** dans le soutien à l'innovation locale et le développement des talents africains, renforçant ainsi son leadership auprès de ses parties prenantes.

Chez Synelia, la **mission** consiste à accompagner les organisations dans leur **transformation numérique** en fournissant des solutions innovantes, durables et personnalisées qui améliorent leurs conditions de travail au quotidien et optimisent leurs performances. Avec ses compétences et ses services, Synelia supporte le développement de la connaissance technologique, la formation et l'employabilité en Afrique, dans une approche inclusive et pluriculturelle. Elle contribue au développement de l'excellence technologique locale.

Partout en Afrique de l'Ouest, Synelia permet à ses clients, ses collaborateurs et à l'écosystème plus large de vivre, travailler et progresser durablement et en toute confiance dans l'espace numérique. Sa mission prioritaire est d'optimiser l'utilisation des nouvelles technologies pour ses clients et de développer avec eux une nouvelle génération de services adaptés aux contextes africains.

L'entreprise a identifié quatre défis majeurs que rencontrent ses clients, quels que soient leur domaine d'activité et leur situation géographique. Elle possède les ressources et les expertises pour les accompagner dans tous les aspects de leur transformation numérique :

- **Réinventer le modèle économique** → grâce à la numérisation et l'automatisation des processus.
- **Améliorer l'expérience client et utilisateur** → via des solutions agiles et centrées sur l'humain.
- **Assurer la sécurité et la confiance** → en protégeant les infrastructures et données contre les cybermenaces.
- **Renforcer l'excellence opérationnelle** → par l'optimisation des infrastructures IT et la formation continue.

Ces engagements se traduisent concrètement par des initiatives comme l'Open Digital Academy, qui forme les talents locaux et contribue à la montée en compétences pour une transformation numérique réussie et inclusive.

III- Schéma de l'organisation de l'entreprise Synelia

Afin de mieux comprendre le contexte organisationnel dans lequel s'inscrit le projet, il est nécessaire de présenter la structure organisationnelle de l'entreprise SYNELIA.

Cette représentation permet d'identifier les différents niveaux hiérarchiques, les services impliqués ainsi que les relations fonctionnelles entre les acteurs intervenant dans le processus de gestion des demandes documentaires.

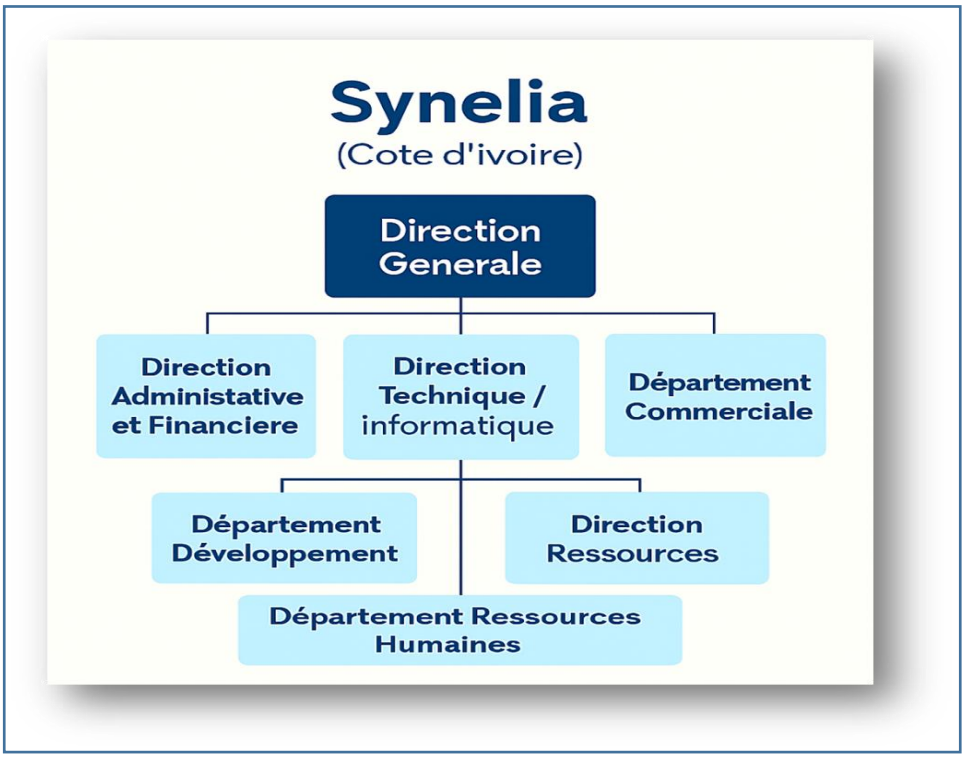


Figure 1 : Schéma de l'organisation de l'entreprise SYNELIA (Côte d'Ivoire)

Ce schéma met en évidence une organisation structurée autour de plusieurs services fonctionnels, notamment les services administratifs, techniques et décisionnels.

Dans le cadre de ce projet, cette organisation a été prise en compte pour définir les rôles des utilisateurs du système, en particulier les profils de demandeur, validateur et administrateur, ainsi que le circuit de validation des documents.

La compréhension de cette structure a permis de concevoir un workflow de validation cohérent avec le fonctionnement réel de l’entreprise.

Chapitre 2 : PRESENTATION DU PROJET

I- Etude de l'existant

1- Présentation de l'existant

La gestion des demandes de documents représente un processus clé dans la plupart des entreprises informatiques comme Synelia. Actuellement, elle est réalisée de façon majoritairement manuelle ou semi-automatisée. À la réception d'une demande (par email, formulaire papier ou verbal), les données relatives à celle-ci (nom du demandeur, type de document, justificatifs) sont saisies dans un fichier Excel ou un tableur partagé, puis conservées numériquement ou physiquement. La validation et la génération du document se font souvent manuellement (rédaction dans Word, signature physique ou électronique basique), et l'archivage dépend de procédures informelles.

Ce système existant repose sur des outils bureautiques standards (Excel, Word, emails), sans plateforme dédiée, ce qui limite la centralisation et l'automatisation.

2- Critique de l'existant

Pour évaluer les limites du système actuel, voici une analyse comparative des fonctionnalités attendues par rapport à la réalité observée chez Synelia (ou dans des contextes similaires d'entreprises IT en Côte d'Ivoire) :

Fonctionnalité attendue	Disponibilité dans l'existant
Connaissance en temps réel de l'état de chaque demande (en cours, validée, rejetée)	Non
Soumission sécurisée et structurée des demandes (formulaires en ligne)	Non
Alertes automatiques aux employés concernés (validation, génération)	Non
Génération automatique de documents en multi-formats (PDF, Word, Excel) avec templates	Non

Traçabilité complète des demandes (historique, audits)	Non
Recherche rapide et archivage centralisé des documents générés	Partielle (via Excel, mais limitée)
Sécurité des données (chiffrement, authentification forte)	Non

Tableau 1: Analyse du système de gestion des demandes de documents actuel chez Synelia.

Comme le montre ce tableau, le système existant présente de nombreuses insuffisances : absence d'automatisation, risques d'erreurs humaines, délais prolongés, manque de traçabilité et vulnérabilités en termes de sécurité et de conformité (ex. : protection des données personnelles).

3- Solution apportée

Au vu des déficiences du système actuel, nous avons décidé de mettre en place une **plateforme web sécurisée pour la gestion automatisée des demandes de documents et la génération multi-formats (PDF, Word, Excel)** chez Synelia. Cette solution full-stack (backend Spring Boot et frontend Angular) permettra d'automatiser les flux, d'assurer une traçabilité complète et une génération rapide de documents à partir de templates prédéfinis.

Dans le cahier des charges, nous détaillerons les besoins fonctionnels et non fonctionnels de cette solution.

II- Cahier de charge

1- Présentation et contexte du sujet

Synelia est une entreprise spécialisée dans la transformation numérique qui fournit des solutions informatiques, des services de conseil IT, d'intégration de systèmes et de cybersécurité à ses clients en Côte d'Ivoire et en Afrique de l'Ouest. Grâce à ses technologies de pointe, ses partenariats stratégiques (Microsoft, HP, Oracle, WALLIX, etc.) et son expertise en digitalisation, Synelia accompagne la

transformation numérique de ses clients dans divers secteurs : finance, télécoms, secteur public, industrie et services.

Dans ce cadre, l'entreprise traite quotidiennement un volume important de demandes de documents administratifs et officiels (attestations de formation, certificats, rapports de projet, contrats, factures, etc.), tant en interne (pour les employés) qu'en externe (pour les clients et partenaires). Ces demandes sont actuellement gérées de manière manuelle ou semi-automatisée, ce qui génère des délais prolongés, des erreurs et un manque de traçabilité.

Afin d'y remédier, nous avons décidé de réaliser une **plateforme web sécurisée de gestion automatisée des demandes de documents et de génération multi-formats (PDF, Word, Excel)**.

2- Objectifs

Ce cahier des charges est rédigé afin de s'assurer que l'application développée réponde pleinement aux besoins en termes de fonctionnalités, de performance et de sécurité pour une solution unique de « Gestion des Demandes de Documents ».

L'objectif principal de ce projet est d'automatiser le système de gestion des demandes de documents chez Synelia afin de résoudre les problèmes majeurs identifiés (délais, erreurs humaines, manque de traçabilité et de sécurité).

L'application devra notamment :

- Permettre la soumission en ligne sécurisée des demandes de documents par les utilisateurs (employés, clients).
- Offrir un état en temps réel du traitement de chaque demande (soumis, en cours de validation, validé, rejeté).
- Assurer une traçabilité complète du cycle de vie de la demande (historique des actions, responsables impliqués).
- Faciliter le suivi et la validation des demandes par les responsables concernés (notifications automatiques).
- Générer automatiquement les documents demandés à partir de templates prédéfinis, en multi-formats (PDF, Word, Excel).

- Permettre la prévisualisation, le téléchargement et l'archivage sécurisé des documents générés.
- Éviter les doublons, les pertes de données et les traitements redondants.
- Fluidifier l'ensemble du processus documentaire tout en garantant la conformité et la sécurité des données.

3- Besoins de l'entreprise

3-1. Besoins fonctionnels

Les besoins fonctionnels désignent ce que l'application devra faire, c'est-à-dire les fonctionnalités principales du système qui sera mis en place. L'application à réaliser comportera plusieurs fonctionnalités clés, parmi lesquelles :

- Gestion des utilisateurs et des rôles (authentification sécurisée, profils : demandeur, validateur, administrateur)
- Soumission de demandes de documents via des formulaires dynamiques et intuitifs (avec upload de pièces jointes si nécessaire).
- Suivi en temps réel de l'état des demandes (soumis, en attente de validation, validé, rejeté, généré).
- Validation multi-niveaux des demandes avec notifications automatiques (email ou in-app) aux responsables concernés.
- Gestion des templates de documents (création, modification et stockage par l'administrateur).
- Génération automatique des documents à partir des données saisies et des templates prédéfinis, en multi-formats (PDF via iText ou PDFBox, Word et Excel via Apache POI).
- Téléchargement sécurisé et archivage centralisé des documents générés avec historique complet.
- Recherche avancée des demandes (par date, statut, demandeur, type de document).
- Tableau de bord administrateur pour superviser l'ensemble des demandes, générer des rapports statistiques et gérer les paramètres du système.

Ces fonctionnalités permettront d'automatiser l'ensemble du flux documentaire, de la soumission à la délivrance, tout en assurant une traçabilité irréprochable.

3-2. Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité du système en termes de performance, sécurité, ergonomie et contraintes techniques. Ils concernent notamment :

- **Sécurité** : Authentification forte (JWT ou OAuth2), chiffrement des données sensibles, gestion fine des autorisations (RBAC), protection contre les attaques courantes (OWASP Top 10), et conformité aux bonnes pratiques de protection des données personnelles.
- **Performances** : Temps de réponse inférieur à 2 secondes pour les opérations courantes, génération de documents PDF en moins de 5 secondes, et capacité à gérer au moins 100 utilisateurs simultanés.
- Scalabilité : Architecture permettant une évolution future (ajout de nouveaux types de documents, intégration avec d'autres systèmes de Synelia).
- Ergonomie et accessibilité : Interface responsive (Angular Material ou similaire), compatible mobile et tablette, respect des normes d'accessibilité WCAG.
- Fiabilité et maintenance : Gestion des erreurs, journalisation des actions (logs), sauvegardes automatiques de la base de données.
- Contraintes techniques :
- Backend : Java avec Spring Boot (version récente).
- Frontend : Angular (version récente).
- Base de données : PostgreSQL ou MySQL.
- Déploiement : Compatible conteneurs (Docker) et cloud (AWS, Azure ou serveur local).
- Système d'exploitation : Multi-plateforme (Linux/Windows pour le serveur).

PARTIE 2 : ETUDE TECNHNIQUE

Chapitre 4 : ÉTUDE ET CHOIX DES SOLUTIONS TECHNIQUES

I- Étude comparative des technologies Backend

Le choix d'une technologie backend pour une plateforme de gestion des demandes documentaires et de génération automatisée de documents est stratégique. Le système doit répondre à des exigences élevées en matière de **sécurité**, **traçabilité**, **génération de documents multi-formats** (PDF, Word, Excel), **scalabilité** et **maintenabilité** dans un contexte d'entreprise.

Nous avons comparé trois solutions matures et largement utilisées dans le développement d'applications d'entreprise :

- **Spring Boot (Java)**
- **Node.js avec Express**
- **Laravel (PHP)**

Ces technologies ont été sélectionnées car elles couvrent un large spectre de philosophies de développement (orienté entreprise, JavaScript fullstack, et productivité web) tout en étant largement utilisées dans le contexte africain et ivoirien.

1- Présentation détaillée des technologies

Spring Boot (Java) :

Spring Boot est un framework open source basé sur le Spring Framework, conçu pour simplifier le développement d'applications Java d'entreprise. Il adopte le principe « convention over configuration » et permet de créer des applications standalone prêtes pour la production. Il intègre nativement un écosystème complet : Spring Security pour l'authentification et l'autorisation, Spring Data JPA pour la persistance, Spring Boot Actuator pour la surveillance, et des intégrations natives avec des bibliothèques de génération de documents telles que **iText**, **PDFBox** et **Apache POI**. Spring Boot est particulièrement apprécié dans les environnements bancaires, télécoms et administrations en Côte d'Ivoire.

Node.js avec Express :

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Google Chrome. Express.js est un framework minimaliste qui permet de construire rapidement des API REST. Son modèle asynchrone non bloquant lui confère une excellente réactivité. Cependant, la gestion des opérations complexes (transactions distribuées, génération de documents lourds) repose souvent sur des bibliothèques tierces (pdf-lib, docx, exceljs) moins matures que leurs équivalents Java.

Laravel (PHP) :

Laravel est un framework PHP suivant l'architecture MVC (Model-View-Controller). Il est réputé pour sa syntaxe élégante, ses fonctionnalités intégrées (Auth, Queue, Notifications, Eloquent ORM) et sa courbe d'apprentissage relativement douce. Bien qu'il soit très productif pour des applications web classiques, il reste moins optimisé pour les traitements lourds de documents et les exigences de sécurité très élevées des systèmes critiques.

2- Critères de comparaison

L'évaluation comparative des technologies backend a été menée de manière structurée et objective. Les critères retenus ne sont pas arbitraires : ils découlent directement des **besoins fonctionnels** (gestion des demandes, workflow de validation, génération multi-formats) et **non fonctionnels** (sécurité, performance, scalabilité, maintenabilité) identifiés dans le cahier des charges du projet.

Chaque critère est détaillé ci-dessous avec son importance dans le contexte du système de gestion documentaire de Synelia :

- **Performance et scalabilité** La plateforme doit pouvoir supporter un nombre croissant d'utilisateurs (employés, responsables et potentiellement clients externes) tout en maintenant des temps de réponse rapides, notamment lors de la génération de documents PDF ou Excel. Une mauvaise performance pourrait entraîner des retards dans le traitement des demandes, nuisant à l'efficacité opérationnelle de l'entreprise.
- **Sécurité et conformité** Les données manipulées (informations personnelles, documents administratifs, historiques de validation) sont sensibles. Le backend doit offrir une authentification forte, une gestion fine des autorisations (RBAC), la protection contre les attaques courantes (OWASP Top 10), le chiffrement des données et la traçabilité des actions pour respecter les exigences de conformité (protection des données personnelles en vigueur en Côte d'Ivoire).

- **Capacité de génération de documents** Cœur du projet : le système doit générer automatiquement des documents en formats PDF (mise en page professionnelle), Word et Excel à partir de templates. Ce critère évalue la robustesse, la fiabilité et la qualité de rendu des bibliothèques disponibles pour chaque technologie.
- **Gestion des transactions et workflows** Le processus implique un workflow multi-niveaux de validation (soumission → validation 1 → validation 2 → génération). Le backend doit garantir l'intégrité des données via des transactions ACID, gérer les rejets, les notifications et maintenir la cohérence même en cas d'erreur ou d'accès concurrent.
- **Maintenabilité et architecture logicielle** En tant que projet de fin d'études destiné à être potentiellement déployé et maintenu par Synelia, l'architecture doit favoriser la lisibilité du code, la modularité, les tests unitaires et l'évolutivité (ajout de nouveaux types de documents ou intégration avec d'autres systèmes de l'entreprise).
- **Écosystème et intégrations** La technologie choisie doit disposer d'un écosystème riche permettant une intégration facile avec des outils tiers (envoi d'emails, stockage de fichiers, bases de données, monitoring) et avec les systèmes existants chez Synelia (Microsoft 365, serveurs d'entreprise, etc.).
- **Adoption dans le contexte local (Côte d'Ivoire / Afrique de l'Ouest)** Il est important de choisir une technologie bien représentée sur le marché local afin de faciliter le recrutement de développeurs, le support technique et la continuité du projet après le stage.
- **Coût de formation et disponibilité des développeurs** Compte tenu du temps limité du stage et des ressources humaines disponibles chez Synelia et à l'ENSIT, la technologie doit permettre une montée en compétence rapide tout en restant valorisante sur le plan professionnel.

Ces critères ont été pondérés selon leur criticité pour le projet : la **sécurité**, la **génération de documents** et la **gestion des workflows** ont reçu une importance majeure.

3- Tableau comparatif

Afin de synthétiser l'analyse, le tableau suivant compare les trois technologies selon les critères définis précédemment. Les appréciations sont basées sur une revue de la documentation officielle, des retours d'expérience dans des projets d'entreprise similaires et des benchmarks disponibles.

Critère	Spring Boot (Java)	Node.js + Express	Laravel (PHP)	Vainqueur
Performance & Scalabilité	Excellente (JVM optimisée, support natif Kubernetes et conteneurs)	Très bonne (modèle asynchrone non bloquant)	Bonne	**Spring Boot**
Sécurité	Excellente (Spring Security, JWT, OAuth2, protection OWASP intégrée)	Bonne (middleware, Passport.js)	Très bonne (système d'authentification intégré)	**Spring Boot**
Génération de Documents	Excellente (iText 7, PDFBox, Apache POI pour PDF/Word/Excel)	Moyenne (pdf-lib, docx, exceljs)	Moyenne (dompdf, PhpSpreadsheet)	**Spring Boot**
Gestion des Transactions & Workflows	Excellente (transactions déclaratives, Spring Batch)	Moyenne (gestion manuelle ou via libraries)	Bonne (Eloquent + Queue)	**Spring Boot**
Traçabilité & Audit	Excellente (AOP, Spring Events, Actuator)	Bonne	Bonne	**Spring Boot**
Maintenabilité long terme	Excellente (typage fort, architecture modulaire, tests facilités)	Moyenne (typage dynamique même avec TypeScript)	Bonne (MVC clair)	**Spring Boot**
Écosystème & Intégrations	Très riche (écosystème Spring complet)	Riche (particulièrement avec JavaScript fullstack)	Moyen à bon	**Spring Boot**
Adoption dans le contexte local (Côte d'Ivoire)	Bonne et en forte croissance dans les grandes entreprises	Très bonne	Très bonne	Node.js / Laravel
Disponibilité des développeurs	Bonne (en croissance grâce aux formations ENSIT et entreprises)	Très bonne	Très bonne	Node.js / Laravel
Courbe d'apprentissage	Moyenne à élevée	Faible	Faible	Laravel / Node.js

Tableau 2: Tableau comparatif des technologies backend.

4- Analyse approfondie des forces et faiblesses

Après la présentation comparative synthétique, cette section propose une analyse plus qualitative et contextualisée de chaque technologie, en lien direct avec les enjeux spécifiques du projet de plateforme de gestion des demandes documentaires chez Synelia.

Spring Boot (Java) :

Forces principales : Spring Boot se distingue par son architecture modulaire et mature, compatible avec des approches avancées telles que l'Architecture Hexagonale et le Domain-Driven Design (DDD). Il offre une sécurité de niveau entreprise grâce à Spring Security, qui permet une implémentation fine des politiques d'authentification (JWT, OAuth2) et d'autorisation (RBAC). Sa capacité à intégrer nativement des bibliothèques puissantes comme **iText 7**, **PDFBox** et **Apache POI** en fait la solution idéale pour la génération de documents administratifs complexes nécessitant une mise en page professionnelle, des en-têtes, pieds de page, signatures numériques et tableaux dynamiques.

De plus, Spring Boot excelle dans la gestion des transactions déclaratives et des workflows longs, ce qui est essentiel pour le circuit de validation multi-niveaux des demandes. Son écosystème riche (Spring Data JPA, Spring Events, Actuator) facilite la traçabilité, l'audit et la surveillance de l'application. Enfin, Java bénéficie d'une forte crédibilité auprès des grandes entreprises et institutions publiques en Côte d'Ivoire, ce qui facilite l'adhésion et la pérennité de la solution au sein de Synelia.

Faiblesses : La principale contrainte réside dans une courbe d'apprentissage plus élevée, particulièrement pour les développeurs habitués aux frameworks web légers. Le temps de démarrage du projet peut être légèrement plus long en raison de la configuration initiale et de la verbosité du langage Java. Cependant, ces inconvénients sont largement compensés par la robustesse et la maintenabilité à long terme.

Node.js avec Express :

Forces principales : Node.js offre une très grande rapidité de développement et une excellente productivité, grâce à son modèle asynchrone non bloquant et à l'utilisation d'un seul langage (JavaScript/TypeScript) pour le backend et le frontend Angular. Cela permet un partage de modèles et une communication fluide entre les deux couches. Il est particulièrement performant pour les opérations d'entrées/sorties intensives (upload de pièces jointes, notifications en temps réel).

Faiblesses : Malgré l'apport de TypeScript, le typage reste moins rigoureux qu'en Java, ce qui peut entraîner des erreurs difficiles à détecter dans un système critique. La gestion des transactions distribuées et des workflows complexes (validation multi-niveaux avec rollback) est moins naturelle et souvent réalisée de manière manuelle ou via des bibliothèques tierces moins matures. Par ailleurs, les outils de génération de documents (pdf-lib, exceljs, docx) sont moins puissants et moins stables que leurs homologues Java, particulièrement pour des documents officiels nécessitant une mise en forme stricte et une conformité réglementaire.

Laravel (PHP) :

Forces principales : Laravel est reconnu pour sa productivité exceptionnelle et sa syntaxe élégante. Il propose de nombreux outils intégrés (Artisan CLI, Queue system, Horizon, Telescope) qui accélèrent considérablement le développement. Sa communauté est très active en Afrique de l'Ouest, ce qui facilite le recrutement et le support. Le framework offre également une bonne gestion des notifications et des tâches asynchrones.

Faiblesses : Sous forte charge concurrente, Laravel montre des limites en termes de performance par rapport à la JVM de Java. Son écosystème est moins adapté à la génération de documents administratifs complexes (attestations, contrats, rapports avec mise en page professionnelle et signatures). De plus, PHP est parfois perçu comme moins « entreprise » dans les grands groupes ivoiriens et les institutions financières, ce qui pourrait limiter la crédibilité et l'évolutivité future de la solution chez Synelia.

5- Conclusion de l'étude comparative

Au terme de cette analyse multicritère approfondie, **Spring Boot (Java)** se distingue nettement comme la technologie backend la plus adaptée à la conception et à l'implémentation de la plateforme de gestion automatisée des demandes documentaires chez Synelia.

Ce choix est motivé par plusieurs facteurs décisifs. D'une part, Spring Boot excelle dans les domaines critiques du projet : la **sécurité de niveau entreprise**, la **gestion robuste des workflows de validation multi-niveaux**, et surtout la **génération de documents multi-formats** (PDF, Word, Excel) grâce à un écosystème mature (iText, PDFBox, Apache POI). Ces capacités répondent directement aux exigences de traçabilité, de conformité et de professionnalisme des documents générés.

D'autre part, l'architecture modulaire, la forte typisation et les outils de monitoring intégrés de Spring Boot offrent une **maintenabilité et une évolutivité** supérieures, essentielles pour une solution destinée à être déployée dans un environnement professionnel et potentiellement étendue à d'autres processus métier de Synelia.

Bien que Node.js + Express et Laravel présentent des avantages indéniables en termes de rapidité de développement et de productivité immédiate, ils montrent des limites significatives dans la gestion de documents complexes, la robustesse transactionnelle et la crédibilité entreprise, particulièrement dans le contexte ivoirien où les grands comptes et institutions privilégient des technologies éprouvées comme Java.

Ce choix s'inscrit parfaitement dans la stratégie technologique de Synelia, entreprise leader dans la transformation numérique en Côte d'Ivoire. Il facilite non seulement l'intégration avec les systèmes existants (Microsoft, Oracle, etc.), mais renforce également la valeur ajoutée du projet en développant des compétences hautement recherchées sur le marché de l'emploi local et régional.

En conclusion, Spring Boot représente le meilleur équilibre entre performance, sécurité, fiabilité et pérennité, tout en répondant aux ambitions d'excellence et d'innovation de Synelia.

II- Étude comparative des technologies Frontend

Le choix de la technologie frontend revêt une importance capitale dans la réussite d'une plateforme web interactive telle que celle développée pour Synelia. En effet, l'interface utilisateur constitue le principal point de contact entre les utilisateurs (demandeurs, validateurs et administrateurs) et le système. Elle doit offrir une **expérience utilisateur fluide et intuitive**, une **gestion performante des formulaires dynamiques** (sélection de types de documents, champs conditionnels, upload de pièces jointes), une **navigation claire** dans les workflows de validation multi-niveaux, ainsi qu'une **intégration robuste et sécurisée** avec le backend Spring Boot.

Dans le cadre de ce projet, le frontend doit également garantir une **responsivité** parfaite (compatible desktop, tablette et mobile), une **accessibilité** conforme aux bonnes pratiques, et une maintenabilité élevée afin de faciliter les évolutions futures de la plateforme.

Pour répondre à ces exigences, nous avons réalisé une étude comparative entre les trois frameworks JavaScript les plus matures et les plus utilisés en 2025 :

- **Angular** (développé par Google)
- **React** (développé par Meta)
- **Vue.js**

Ces frameworks ont été sélectionnés car ils dominent le marché du développement frontend moderne et couvrent des approches différentes : framework complet et structuré (Angular), bibliothèque flexible (React) et framework progressif (Vue.js).

1- Présentation détaillée des technologies

Angular (Google) :

Développé et maintenu par Google depuis 2016, Angular est un **framework frontend complet et opinionated**. Il repose sur une architecture solide basée sur des composants, des modules et des services. Angular impose l'utilisation de **TypeScript** par défaut, ce qui apporte un typage statique fort et une meilleure maintenabilité. Il propose une suite d'outils intégrés et cohérents : système de routing avancé, Reactive Forms pour la gestion de formulaires complexes, HttpClient pour les communications avec le backend, RxJS pour la programmation réactive, et Angular Material pour un design system professionnel.

Ce framework est particulièrement adapté aux **applications d'entreprise de grande envergure**, où la structuration du code, la scalabilité et la cohérence architecturale sont primordiales.

React (Meta) :

Créée en 2013 par Meta (anciennement Facebook), React est une **bibliothèque JavaScript** plutôt qu'un framework complet. Elle se concentre sur la construction d'interfaces utilisateur déclaratives et composables grâce à son célèbre Virtual DOM. Sa grande force réside dans sa **flexibilité** extrême. React est généralement complété par un ensemble d'outils additionnels tels que React Router (routing), TanStack Query (gestion des données serveur), Redux ou Zustand (state management), et des bibliothèques de composants comme Material-UI ou Ant Design.

Cette approche « à la carte » en fait une solution très prisée pour les projets nécessitant une grande liberté créative et une évolution rapide.

Vue.js :

Créé par Evan You en 2014, Vue.js est un **framework progressif et léger**. Il se distingue par sa courbe d'apprentissage particulièrement douce et sa syntaxe intuitive. Vue.js peut être utilisé de manière incrémentale, soit comme une simple bibliothèque pour enrichir des pages existantes, soit comme un framework complet avec Vue Router, Pinia (state management) et Vuetify ou Quasar pour les composants UI.

Il offre un excellent équilibre entre performance, simplicité et flexibilité, ce qui en fait un choix populaire pour les projets de taille moyenne.

2- Critères de comparaison

L'évaluation comparative des technologies frontend a été réalisée de manière structurée et objective. Les critères retenus découlent directement des besoins fonctionnels et non fonctionnels identifiés dans le cahier des charges du projet, notamment la gestion de formulaires dynamiques, le suivi des workflows de validation, l'expérience utilisateur intuitive et l'intégration avec le backend Spring Boot.

Chaque critère est détaillé ci-dessous avec sa justification dans le contexte de la plateforme de gestion des demandes documentaires chez Synelia :

- **Architecture et structuration du code** Le frontend doit offrir une architecture claire, modulaire et maintenable, afin de faciliter le développement, les tests et les évolutions futures (ajout de nouveaux types de documents ou de fonctionnalités). Une bonne structuration réduit les risques de dette technique dans un projet de fin d'études destiné à un environnement professionnel.
- **Gestion des formulaires complexes** Cœur métier du système : les utilisateurs doivent pouvoir soumettre des demandes via des formulaires dynamiques (champs conditionnels selon le type de document, upload de pièces jointes, validation en temps réel). Ce critère évalue la puissance et la simplicité des outils de gestion de formulaires proposés par chaque technologie.
- **Performance et rendu** L'application doit offrir une navigation fluide, un rendu rapide des tableaux de bord et une bonne réactivité lors des changements d'état des demandes (soumis → en validation → validé). Une excellente performance est essentielle pour une bonne adoption par les utilisateurs finaux.
- **Intégration avec le backend (Spring Boot)** La communication avec l'API REST (authentification JWT, récupération des demandes, soumission de formulaires, upload de fichiers) doit être simple, sécurisée et efficace. Ce critère mesure la qualité des clients HTTP, des intercepteurs et de la gestion des erreurs.
- **Bibliothèques de composants UI** L'interface doit être moderne, responsive et professionnelle (tableaux, modals, notifications, formulaires élégants). La disponibilité d'un design system mature accélère le développement tout en garantissant une cohérence visuelle.
- **State Management et réactivité** Le système doit gérer en temps réel l'état des demandes, les notifications et les workflows. Ce critère évalue la facilité et la robustesse de la gestion de l'état global de l'application.

- **Maintenabilité et scalabilité** La solution choisie doit permettre une maintenance aisée et une évolution future (nouveaux modules, multi-langues, accessibilité). La typage fort et l'organisation du code sont des facteurs déterminants.
- **Courbe d'apprentissage et productivité** Compte tenu du délai du stage (environ 8 mois), la technologie doit permettre une montée en compétence rapide tout en restant pertinente sur le plan professionnel.
- **Adoption dans le contexte professionnel ivoirien** Il est important de privilégier une technologie bien représentée sur le marché local (entreprises, cabinets de recrutement, partenaires de Synelia) pour favoriser la valorisation du projet et une éventuelle poursuite de son développement après le stage.

Ces critères ont été pondérés en accordant une importance particulière à la **gestion des formulaires**, à **l'architecture** et à **l'intégration backend**, éléments critiques pour la réussite de la plateforme.

3- Tableau comparatif

Le tableau suivant synthétise l'analyse comparative des trois frameworks frontend selon les critères définis précédemment. Les évaluations sont basées sur l'expérience de développement, la documentation officielle et les retours d'usage dans des projets d'application d'entreprise similaires.

Critère	Angular	React	Vue.js	Vainqueur
Architecture & Structuration	Excellente (framework opinionated, CLI puissant, modules)	Bonne (flexible mais nécessite choix d'outils)	Très bonne (progressive et flexible)	**Angular**
Gestion des Formulaires Complexes	Excellente (Reactive Forms + validation native)	Bonne (React Hook Form + validation)	Bonne	**Angular**
Performance et Rendu	Très bonne	Excellente (Virtual DOM + optimisation)	Excellente	React / Vue
Intégration Backend (Spring Boot)	Excellente (HttpClient + Interceptors + Auth)	Excellente (Axios + TanStack Query)	Très bonne	**Angular**
Composants UI / Design System	Excellente (Angular Material)	Excellente (Material-UI, Ant Design, Shadcn)	Très bonne (Vuetify, Quasar, PrimeVue)	Angular / React
State Management & Réactivité	Bonne (Signals + NgRx)	Excellente (Zustand, Redux, Context API)	Excellente (Pinia)	React / Vue

Maintenabilité long terme	Excellente (typage fort + architecture imposée)	Bonne (dépend de la discipline de l'équipe)	Très bonne	**Angular**
Support TypeScript	Natif et obligatoire	Bon (via TypeScript)	Bon	**Angular**
Courbe d'apprentissage	Moyenne à élevée	Moyenne	Faible	Vue.js
Adoption en entreprise (Côte d'Ivoire)	Bonne et en forte croissance	Très bonne	Moyenne à bonne	**React**
Adaptation au projet documentaire	Excellente	Très bonne	Bonne	**Angular**

Tableau 3: Tableau comparatif des technologies frontend.

4- Analyse approfondie des forces et faiblesses

Après la présentation comparative synthétique, cette section propose une analyse plus qualitative et contextualisée de chaque technologie frontend, en lien direct avec les enjeux spécifiques du projet de plateforme de gestion des demandes documentaires chez **Synelia**.

L'interface utilisateur joue un rôle central dans l'adoption de la solution par les utilisateurs (demandeurs, validateurs et administrateurs). Elle doit allier intuitivité, performance, gestion de formulaires complexes et cohérence avec le backend Spring Boot, tout en respectant les contraintes de temps et de maintenabilité d'un projet de fin d'études.

Angular :

Forces principales : Angular se distingue par son architecture complète, cohérente et fortement structurée, ce qui en fait un choix particulièrement adapté aux applications d'entreprise complexes comme la plateforme de gestion des demandes documentaires. Son système de **Reactive Forms** est l'un des plus puissants du marché, permettant de gérer facilement des formulaires dynamiques avec champs conditionnels, validations en temps réel, et upload de pièces jointes selon le type de document. L'utilisation native de **TypeScript** renforce considérablement la robustesse du code et facilite la détection précoce des erreurs.

De plus, **Angular Material** offre un design system professionnel, moderne et responsive prêt à l'emploi, garantissant une interface utilisateur de qualité entreprise. L'intégration avec Spring Boot est particulièrement fluide grâce à **HttpClient**, aux **Interceptors** (pour la gestion centralisée des JWT et

des erreurs) et à RxJS pour la gestion asynchrone des workflows de validation. Cette cohérence globale facilite la maintenabilité et l'évolutivité du projet.

Faiblesses : La courbe d'apprentissage est plus raide en raison de sa richesse conceptuelle (modules, services, dependency injection, RxJS, etc.). Le framework étant relativement verbeux, le temps de développement initial peut être plus long. Par ailleurs, le bundle de production peut être plus lourd si les bonnes pratiques d'optimisation (lazy loading, tree shaking) ne sont pas rigoureusement appliquées.

React :

Forces principales : React offre une très grande flexibilité et une excellente performance grâce à son Virtual DOM optimisé. Son écosystème extrêmement riche permet de créer des interfaces hautement dynamiques et modernes. La communauté étant la plus importante, il est facile de trouver des solutions aux problèmes rencontrés. Avec des outils comme React Hook Form, TanStack Query et Material-UI, il est possible de développer rapidement des interfaces réactives et agréables.

Faiblesses : Contrairement à Angular, React n'impose pas d'architecture stricte. Le développeur doit choisir et assembler lui-même les outils (routing, state management, forms, etc.), ce qui peut conduire à une architecture moins cohérente, surtout dans le cadre d'un projet académique réalisé en temps limité. Cette liberté peut également compliquer la maintenabilité à long terme et augmenter le risque de dette technique.

Vue.js :

Forces principales : Vue.js brille par sa syntaxe simple, intuitive et progressive, offrant une excellente courbe d'apprentissage. Il permet d'obtenir rapidement des résultats visibles tout en maintenant une très bonne performance. Son système de réactivité et **Pinia** (state management) sont élégants et faciles à maîtriser. Il représente un bon compromis entre productivité et performance.

Faiblesses : Bien que très agréable à utiliser, Vue.js est moins adopté dans les grandes entreprises et institutions en Côte d'Ivoire par rapport à Angular et React. Son écosystème, bien que de qualité, reste moins riche pour les applications très complexes nécessitant une forte structuration et une scalabilité importante, comme une plateforme de gestion documentaire avec workflows élaborés.

5- Conclusion de l'étude comparative

Au terme de cette analyse multicritère approfondie, **Angular** se révèle être le choix le plus pertinent et le plus adapté pour le développement du frontend de la plateforme de gestion automatisée des demandes documentaires chez Synelia.

Ce choix est principalement motivé par la **structure rigoureuse et cohérente** qu'offre Angular, son excellente gestion des formulaires complexes via Reactive Forms, ainsi que son intégration fluide et sécurisée avec le backend Spring Boot (via HttpClient et Interceptors). Ces atouts sont déterminants dans un projet où l'interface doit permettre une soumission intuitive des demandes, un suivi visuel clair des workflows de validation et une expérience utilisateur professionnelle.

En outre, l'utilisation native de TypeScript, combinée à une architecture modulaire bien définie, garantit une **maintenabilité élevée** et une **scalabilité** future, aspects essentiels pour une application destinée à un environnement d'entreprise comme Synelia. Angular Material permet également de livrer rapidement une interface moderne, responsive et cohérente, renforçant ainsi l'adoption de la solution par les utilisateurs finaux.

Bien que React et Vue.js offrent des avantages notables en termes de flexibilité et de courbe d'apprentissage, ils présentent des limites en matière de structuration imposée et de cohérence architecturale, particulièrement dans le cadre contraint d'un projet de fin d'études. Angular offre le meilleur équilibre entre productivité maîtrisée, robustesse et professionnalisme.

Ce choix technologique renforce la cohérence globale du stack (Spring Boot + Angular), produit un code robuste, fortement typé et conforme aux standards exigés dans un mémoire d'ingénieur en Sciences Informatiques. Il valorise également les compétences acquises, hautement recherchées sur le marché de l'emploi en Côte d'Ivoire et dans les entreprises de transformation numérique comme Synelia.

III- Étude comparative des méthodes de conception (MERISE vs UML)

Le choix d'une méthode de conception constitue une étape déterminante dans la modélisation d'un système d'information. Il conditionne non seulement la qualité de l'analyse, mais aussi la clarté de la communication entre les acteurs techniques et métiers, ainsi que la facilité de passage de la conception à l'implémentation.

Dans le cadre de ce projet de **plateforme web de gestion des demandes documentaires et de génération automatisée de documents** chez Synelia, le système repose sur un **modèle de données relationnel complexe**, des **workflows de validation multi-niveaux**, une forte **traçabilité** des actions et une exigence élevée de **conformité** et d'**audit**. Ces caractéristiques imposent une méthode de conception capable de modéliser rigoureusement les données tout en restant accessible aux utilisateurs métiers de l'entreprise.

Pour répondre à ces enjeux, nous avons réalisé une étude comparative entre les deux méthodes les plus utilisées dans les projets de systèmes d'information :

- **MERISE** (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise)
- **UML** (Unified Modeling Language)

1- Présentation des méthodes

MERISE :

MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise) est une méthode de conception française développée dans les années 1970-1980 et largement adoptée dans les pays francophones, notamment en France, en Belgique, en Suisse et en Afrique de l'Ouest. Elle est particulièrement adaptée à la conception de **systèmes d'information relationnels** dans un contexte organisationnel.

MERISE se caractérise par une approche **séquentielle et structurée**, qui distingue clairement la **logique des données** de la **logique des traitements**. Elle repose sur plusieurs modèles complémentaires et articulés :

- **Modèle Conceptuel des Données (MCD)** : représentation abstraite des entités et associations.
- **Modèle Logique des Données (MLD)** : traduction en schéma relationnel normalisé.
- **Modèle Physique des Données (MPD)** : implémentation concrète dans un SGBD.
- **Modèle Conceptuel des Traitements (MCT)** et **Modèle Organisationnel des Traitements (MOT)** : modélisation des processus et des flux.

Cette séparation méthodologique offre une grande rigueur dans l'analyse et facilite la communication avec les utilisateurs métiers.

UML :

UML (Unified Modeling Language) est un **langage de modélisation objet standardisé** internationalement par l'OMG (Object Management Group) depuis 1997. Contrairement à MERISE qui est une méthode, UML est avant tout un **langage graphique** très généraliste. Il propose une quinzaine de diagrammes regroupés en deux catégories principales : les diagrammes structurels (classes, objets, composants, déploiement) et les diagrammes comportementaux (cas d'utilisation, séquences, activités, états, etc.).

UML est particulièrement adapté à la **conception orientée objet**, à la modélisation d'architectures logicielles complexes et aux approches agiles. Il est très utilisé dans le développement de systèmes orientés services, d'applications web modernes et dans les environnements utilisant des langages tels que Java, C# ou Python.

2- Critères de comparaison

L'évaluation comparative entre MERISE et UML a été menée de manière objective et structurée. Les critères retenus ne sont pas arbitraires : ils découlent directement des caractéristiques du projet — à savoir un système fortement centré sur un **modèle de données relationnel complexe**, des **workflows de validation multi-niveaux**, une **traçabilité rigoureuse** des actions et une exigence de **clarté** pour les acteurs métiers de Synelia.

Chaque critère est détaillé ci-dessous avec sa justification dans le contexte du projet :

- **Adaptation à la modélisation des données relationnelles** Le système repose sur de nombreuses entités interconnectées (Utilisateur, Demande, TypeDocument, PieceJointe, HistoriqueAction, EtapeValidation, etc.). Ce critère évalue la capacité de chaque méthode à modéliser efficacement ces données tout en respectant les règles de normalisation.
- **Prise en charge des workflows et processus métier** Le processus de gestion des demandes inclut une soumission, une validation multi-niveaux, une génération de documents et une notification. La méthode choisie doit permettre une modélisation claire des flux et des règles de gestion associées.

- **Clarté et lisibilité pour un système documentaire** Les modèles doivent être compréhensibles non seulement par les développeurs, mais aussi par les utilisateurs métiers (responsables, administrateurs) lors des phases de validation avec Synelia.
- **Séparation données / traitements** Une bonne méthode doit distinguer clairement la modélisation des données (statique) de celle des traitements et processus (dynamique), afin d'éviter toute confusion lors de la conception.
- **Niveau de détail pour les bases de données** Le projet nécessite une transformation fluide du modèle conceptuel vers un schéma physique implémentable dans un SGBD (PostgreSQL ou MySQL). Ce critère mesure la précision et la facilité de passage au modèle logique et physique.
- **Adéquation avec le contexte académique et professionnel en Côte d'Ivoire** La méthode doit être bien enseignée dans les écoles d'ingénieurs ivoiriennes (ENSIT, INP-HB, etc.) et reconnue dans les entreprises locales de transformation numérique.
- **Maintenabilité et traçabilité du modèle** Les modèles produits doivent faciliter la maintenance future du système et permettre une traçabilité complète des règles de gestion tout au long du cycle de vie du projet.
- **Facilité d'implémentation technique** La méthode doit permettre un passage relativement direct entre la conception et l'implémentation (Java Spring Boot + Angular + SGBD relationnel).

Ces critères ont été pondérés en accordant une importance majeure à la **modélisation des données relationnelles**, à la **clarté métier** et à l'**adéquation académique/professionnelle locale**.

3- Tableau comparatif

Le tableau suivant synthétise l'analyse comparative entre MERISE et UML selon les critères définis précédemment. Les appréciations sont fondées sur l'expérience de modélisation, les bonnes pratiques académiques et les retours d'usage dans des projets de systèmes d'information similaires.

Critère	MERISE	UML	Vainqueur
Modélisation des données relationnelles	Excellente (MCD, MLD, MPD avec normalisation)	Bonne (Diagramme de classes)	**MERISE**
Gestion des workflows et processus métier	Très bonne (MCT et MOT)	Excellente (Diagrammes d'activités, séquences, états)	**UML**

Clarté et lisibilité pour les non-informaticiens	Excellente (représentation graphique simple et intuitive)	Moyenne (diagrammes plus techniques)	**MERISE**
Séparation Données / Traitements	Excellente (séparation claire et méthodologique)	Moyenne (approche plus intégrée)	**MERISE**
Niveau de détail pour les bases de données	Excellente (passage direct MCD → MLD → MPD)	Bon (Diagramme de classes + notes)	**MERISE**
Adéquation au contexte francophone / Côte d'Ivoire	Excellente (très enseignée et utilisée localement)	Bonne (standard international)	**MERISE**
Orienté Objet et Architecture Logicielle	Moyenne	Excellente (diagrammes de composants, déploiement, etc.)	**UML**
Traçabilité et Audit du modèle	Très bonne (règles de gestion explicites)	Bonne	**MERISE**
Facilité pour un projet de fin d'études	Excellente (démarche pédagogique et structurée)	Bonne (plus généraliste et parfois plus lourd)	**MERISE**
Adaptation globale au projet documentaire	Excellente	Bonne	**MERISE**

Tableau 4: Tableau comparatif des types de modélisation.

4- Analyse approfondie des forces et faiblesses

MERISE

Forces principales : MERISE est particulièrement puissante pour la modélisation des systèmes d'information centrés sur les **données relationnelles**, ce qui correspond parfaitement aux besoins de ce projet. Avec de nombreuses entités interconnectées (Utilisateur, Demande, TypeDocument, PieceJointe, HistoriqueAction, EtapeValidation, Notification, etc.), MERISE permet une représentation claire, normalisée et rigoureuse des données grâce à ses modèles MCD, MLD et MPD.

Sa démarche méthodique et pédagogique facilite la **séparation entre la logique des données et celle des traitements**, tout en produisant des schémas (notamment les MCD) très accessibles aux utilisateurs métiers. Cela constitue un atout majeur pour la phase de validation avec les responsables de Synelia. Par ailleurs, MERISE étant largement enseignée dans les institutions ivoiriennes (ENSIT, INP-HB, etc.) et utilisée dans les entreprises locales, elle offre une excellente adéquation au contexte académique et professionnel du pays.

Faiblesses : MERISE est moins adaptée à la modélisation fine des interactions dynamiques et des aspects comportementaux complexes. Elle offre moins de diagrammes pour représenter les workflows

détaillés ou les architectures orientées objet modernes, ce qui peut nécessiter le recours à d'autres notations en complément.

UML

Forces principales : UML est un langage de modélisation très complet et flexible qui permet de couvrir à la fois les aspects **statiques** (diagramme de classes) et **dynamiques** (diagrammes de séquences, d'activités, d'états-transitions). Il est particulièrement utile pour représenter les workflows de validation multi-niveaux, les cas d'utilisation et les interactions entre les différents acteurs du système (demandeur, validateur, administrateur). Son caractère standardisé internationalement facilite la communication dans des environnements multi-équipes ou agiles.

Faiblesses : Pour un système fortement centré sur les données relationnelles comme celui-ci, les diagrammes de classes UML sont moins intuitifs et moins précis que les MCD MERISE pour exprimer les cardinalités, les dépendances fonctionnelles et les règles de gestion. De plus, UML peut rapidement devenir volumineux et complexe pour un projet de taille moyenne, avec un risque de sur-modélisation. Enfin, ses diagrammes sont parfois moins accessibles aux utilisateurs métiers non-techniques, ce qui peut compliquer les phases de revue avec l'entreprise.

5- Conclusion de l'étude comparative

Au terme de cette étude comparative, **MERISE** a été retenue comme **méthode principale de conception** pour le système d'information de la plateforme de gestion des demandes documentaires chez Synelia.

Ce choix s'explique par plusieurs facteurs déterminants. MERISE offre une **excellente maîtrise de la modélisation des données relationnelles**, une **séparation claire entre les données et les traitements**, ainsi qu'une **grande lisibilité** des modèles par les acteurs métiers. Ces qualités sont particulièrement adaptées à un projet qui repose sur un modèle de données complexe (demandes, types de documents, pièces jointes, workflows de validation, historique et traçabilité).

De plus, MERISE présente une **forte adéquation** avec le contexte académique ivoirien et les pratiques des entreprises locales de transformation numérique. Sa démarche structurée et pédagogique facilite non seulement la conception rigoureuse du système, mais aussi sa validation auprès de l'entreprise et sa transformation progressive vers l'implémentation technique (MLD puis MPD).

Bien qu'**UML** soit un langage puissant et plus généraliste, particulièrement adapté à la modélisation des aspects dynamiques et orientés objet, il s'avère moins optimal pour la modélisation fine et normalisée d'un système de gestion documentaire fortement centré sur les données relationnelles.

UML sera toutefois utilisée en **complément** pour modéliser certains aspects comportementaux du système, tels que les diagrammes de cas d'utilisation, les diagrammes de séquences pour les workflows de validation, ou les diagrammes d'activités.

En conclusion, le choix de MERISE comme méthode principale garantit une **conception rigoureuse, normalisée, traçable et évolutive**, tout en restant cohérente avec les exigences d'un mémoire de fin d'études d'ingénieur en Sciences Informatiques.

IV- Justification des choix retenus

Au terme des études comparatives menées dans ce chapitre, les choix technologiques et méthodologiques suivants ont été retenus pour la réalisation de la plateforme de gestion des demandes documentaires chez Synelia :

- **Backend** : **Spring Boot (Java)**
- **Frontend** : **Angular**
- **Méthode de conception** : **MERISE**

Justification globale des choix

Ces trois choix forment un **stack cohérent, robuste et professionnel** parfaitement adapté aux exigences du projet. Ils répondent à la fois aux contraintes techniques (sécurité, performance, génération de documents), aux besoins fonctionnels (workflow de validation, formulaires dynamiques, traçabilité) et au contexte académique et professionnel ivoirien.

1- Justification du choix du Backend : Spring Boot

Spring Boot a été sélectionné après comparaison avec Node.js + Express et Laravel. Il offre le meilleur compromis sur les critères critiques du projet :

- Excellente **sécurité** avec Spring Security (JWT, OAuth2, RBAC)

- Capacités avancées de **génération de documents** (iText, PDFBox, Apache POI)
- Gestion robuste des **transactions** et **workflows** multi-niveaux
- Architecture modulaire et forte maintenabilité à long terme

Ce choix s'aligne également avec les technologies utilisées par les grands partenaires de Synelia (Microsoft, Oracle, etc.) et renforce la crédibilité de la solution dans un environnement d'entreprise.

2- Justification du choix du Frontend : Angular

Angular a été préféré à React et Vue.js pour les raisons suivantes :

- Architecture **rigoureuse et opinionated** qui garantit une structuration claire du code
- Gestion puissante des **formulaires complexes** via Reactive Forms
- Intégration native et sécurisée avec Spring Boot (HttpClient + Interceptors)
- Utilisation native de **TypeScript**, favorisant la robustesse et la maintenabilité
- Angular Material pour une interface professionnelle et responsive

Ce choix permet de produire un frontend cohérent avec le backend, tout en respectant les standards exigés dans un projet d'ingénieur.

3- Justification du choix de la méthode MERISE

MERISE a été retenue comme méthode principale de conception car elle excelle dans :

- La **modélisation des données relationnelles** (MCD, MLD, MPD)
- La **clarté** des modèles pour les utilisateurs métiers
- La **traçabilité** des règles de gestion
- Son adéquation au **contexte académique et professionnel ivoirien**

UML sera utilisée de manière complémentaire pour modéliser les aspects dynamiques (workflows et interactions utilisateurs).

Cohérence globale du stack retenu

L’association **Spring Boot + Angular + MERISE** constitue un stack moderne, sécurisé et maintenable. Il permet une transition fluide entre la phase de conception (MERISE) et l’implémentation technique (Java + TypeScript). Ce choix reflète non seulement une analyse rigoureuse des alternatives, mais aussi une volonté d’allier **robustesse entreprise**, **productivité** et **excellence académique**.

Ces technologies et méthodes renforcent la valeur ajoutée du projet et démontrent une maîtrise des outils et approches les plus adaptés à un système de gestion documentaire dans le contexte de la transformation numérique en Côte d’Ivoire.

Chapitre 5 : CONCEPTION DU SYSTÈME D'INFORMATION

I- Introduction

La conception d'un système d'information constitue une étape essentielle dans le développement d'une application informatique. Elle permet de définir de manière structurée l'organisation des données ainsi que les différents traitements nécessaires au fonctionnement du système. Dans le cadre de ce projet de dématérialisation et de gestion des documents administratifs, une démarche méthodologique rigoureuse a été adoptée afin de garantir la cohérence, la fiabilité et l'évolutivité de la solution proposée.

Pour mener cette étude, nous avons retenu la méthode MERISE, reconnue pour son efficacité dans la modélisation des systèmes d'information et particulièrement adaptée aux applications reposant sur des bases de données relationnelles. Cette méthode permet de distinguer les aspects liés aux données de ceux liés aux traitements, facilitant ainsi l'analyse et la conception du système.

La démarche suivie dans ce chapitre comprend l'élaboration des règles de gestion, la construction du dictionnaire des données, l'identification des dépendances fonctionnelles, l'établissement de la structure d'accès théorique, la modélisation des données à travers les modèles conceptuel, logique et physique, ainsi que la modélisation des traitements à l'aide des modèles conceptuel et organisationnel des traitements.

L'ensemble de ces travaux permettra de disposer d'une représentation claire et complète du système avant son implémentation à l'aide des technologies retenues dans le chapitre précédent.

II- Règles de gestion

Les règles de gestion définissent les contraintes fonctionnelles et organisationnelles qui régissent le fonctionnement du système de gestion dématérialisée des documents administratifs. Elles traduisent les besoins métier exprimés lors de l'analyse et constituent la base de la modélisation des données et des traitements.

Gestion des utilisateurs

RG1 : Tout utilisateur doit être authentifié pour accéder aux fonctionnalités du système.

RG2 : Chaque utilisateur possède un profil déterminant ses droits d'accès et ses actions autorisées dans le système.

RG3 : Un utilisateur peut soumettre plusieurs demandes de documents administratifs.

Gestion des demandes

RG4 : Toute demande doit être associée à un type de document existant.

RG5 : Chaque demande doit posséder une référence unique générée automatiquement par le système.

RG6 : Une demande doit obligatoirement être créée par un utilisateur identifié.

RG7 : Une demande ne peut être soumise que lorsque tous les champs obligatoires du formulaire ont été correctement renseignés.

Gestion des pièces justificatives

RG8 : Les pièces justificatives obligatoires définies pour un type de document doivent être fournies avant la soumission de la demande.

RG9 : Les fichiers transmis doivent respecter les formats et tailles autorisés par le système.

Gestion du workflow de validation

RG10 : Chaque type de document possède un circuit de validation composé d'une ou plusieurs étapes.

RG11 : Une étape de validation ne peut être exécutée que par un utilisateur habilité.

RG12 : Une demande ne peut passer à l'étape suivante qu'après validation de l'étape en cours.

RG13 : Une demande est considérée comme validée uniquement lorsque toutes les étapes du workflow ont été approuvées.

RG14 : Une demande peut être rejetée à n'importe quelle étape du processus de validation conformément aux règles définies par l'organisation.

Gestion des notifications

RG15 : Le système doit notifier les utilisateurs concernés lors de chaque changement d'état d'une demande.

RG16 : Les notifications sont conservées afin de permettre leur consultation ultérieure.

Gestion des traces et de l'audit

RG17 : Toute action réalisée sur une demande doit être enregistrée dans l'historique du système.

RG18 : Les opérations sensibles doivent être conservées dans un journal d'audit afin de garantir la traçabilité des actions effectuées.

Gestion des données

RG19 : Les informations enregistrées dans le système doivent respecter les règles d'intégrité définies par le modèle de données.

RG20 : Les données relatives aux demandes et aux documents doivent être conservées conformément aux politiques de gestion documentaire de l'organisation.

Les règles de gestion ci-dessus permettent de formaliser les contraintes fonctionnelles du système et de préciser les cardinalités entre les différentes entités identifiées lors de l'analyse.
Elles assurent la cohérence globale du système d'information et constituent une base essentielle pour la construction du Modèle Conceptuel des Données (MCD).

L'ensemble de ces règles a été établi conformément au cahier des charges fonctionnel et traduit fidèlement le cycle de vie d'une demande documentaire, depuis sa création par l'utilisateur jusqu'à sa validation finale, en passant par les différentes étapes de contrôle, de notification et d'audit.

III- Dictionnaire des données

Le dictionnaire des données recense de manière unitaire et exhaustive l'ensemble des propriétés utilisées dans le système. Conformément aux règles formelles de la méthode Merise, les données sont présentées au sein d'une liste plate, épurée de toute structure relationnelle (aucune clé étrangère) et classée par ordre alphabétique.

Légende des types :

- **Type :** N (Numérique), A (Alphabétique), AN (Alphanumérique), B (Booléen), DT (Date/Heure).

Code	Libellé	Type	Longueur
idTypeDocument	Identifiant unique du type de document	N	10
nomTypeDocument	Nom du type de document	A	100
descriptionTypeDocument	Description du type de document	AN	255
categorieTypeDocument	Catégorie du document	A	50
serviceTypeDocument	Service concerné	A	50
modelePdf	Modèle utilisé pour la génération du document	AN	255
idPieceRequise	Identifiant unique de la pièce requise	N	10
nomPieceRequise	Nom de la pièce requise	A	100
descriptionPieceRequise	Description de la pièce	AN	255
obligatoire	Indique si la pièce est obligatoire	B	1
formatAutorise	Formats de fichiers autorisés	A	50
tailleMaxMo	Taille maximale autorisée (Mo)	N	5
idChampFormulaire	Identifiant unique du champ	N	10
nomChampFormulaire	Libellé du champ	A	100
typeChamp	Type du champ (texte, date, liste…)	A	20
obligatoire	Champ obligatoire ou non	B	1
valeurDefaut	Valeur par défaut	AN	255
formatValidation	Règle de validation	AN	100
min	Valeur minimale	N	5
max	Valeur maximale	N	5
idEtapeWorkflow	Identifiant unique de l’étape	N	10
niveauEtapeWorkflow	Niveau de validation	N	2
nomEtapeWorkflow	Nom de l’étape	A	100
delaiMaxJours	Délai maximal en jours	N	3
condition	Condition de passage	AN	255
idDemande	Identifiant unique de la demande	N	10
reference	Référence unique de la demande	A	50
dateCreation	Date de création	DT	—
statut	État courant de la demande	A	30
dateDerniereMaj	Date de dernière mise à jour	DT	—

idValeurChamp	Identifiant unique de la valeur	N	10
valeur	Valeur saisie par l'utilisateur	AN	255
idPieceJointe	Identifiant unique de la pièce jointe	N	10
nomFichier	Nom du fichier	A	150
typeMime	Type MIME du fichier	A	50
tailleOctets	Taille du fichier en octets	N	15
urlStockage	Emplacement de stockage	AN	255
idHistoriqueAction	Identifiant unique de l'action	N	10
dateAction	Date de réalisation de l'action	DT	—
acteur	Auteur de l'action	A	100
action	Action réalisée	A	50
commentaire	Commentaire associé	AN	255
idEtapeValidation	Identifiant unique de l'étape	N	10
niveauEtapeValidation	Niveau de validation	N	2
dateDebut	Date de début	DT	—
dateFin	Date de fin	DT	—
decision	Décision prise	A	30
commentaire	Commentaire du validateur	AN	255
idUtilisateur	Identifiant unique de l'utilisateur	N	10
nomUtilisateur	Nom de l'utilisateur	A	50
prenomUtilisateur	Prénom de l'utilisateur	A	50
emailUtilisateur	Adresse email	A	100
motDePasse	Mot de passe hashé	AN	255
roleUtilisateur	Rôle de l'utilisateur	A	30
serviceUtilisateur	Service de rattachement	A	50
gradeUtilisateur	Grade ou fonction	A	50
idNotification	Identifiant unique de la notification	N	10
canal	Moyen d'envoi	A	20
type	Type de notification	A	50
message	Contenu du message	AN	255
dateEnvoi	Date d'envoi	DT	—
statutEnvoi	Statut d'envoi	A	20
idJournalAudit	Identifiant unique du journal	N	10
dateAction	Date de l'événement	DT	—
acteur	Acteur de l'action	A	100
action	Action réalisée	A	50

details	Détails complémentaires	AN	255

Tableau 5: Dictionnaire Des données du système.

IV- Dépendances fonctionnelles

Les dépendances fonctionnelles permettent d'identifier les relations de détermination entre les différentes propriétés du système. Une propriété B dépend fonctionnellement d'une propriété A (noté A => B) si la connaissance de la valeur de A détermine de manière unique et certaine la valeur de B.

Conformément aux règles de normalisation (respect de la Deuxième et Troisième Forme Normale), les dépendances fonctionnelles du système sont classées de manière exhaustive par identifiant source :

1- Dépendances fonctionnelles issues de la configuration documentaire

- idTypeDocument ==> nomDocument, descriptionDocument, categorieDocument, serviceDocument, modelePdf
- idPieceRequise ==> nomPieceRequise, descriptionPiece, obligatoirePiece, formatAutorise, tailleMaxMo, idTypeDocument
- idChampFormulaire ==> nomChamp, typeChampFormulaire, obligatoireChamp, valeurDefaut, formatValidation, minChamp, maxChamp, idTypeDocument
- idEtapeWorkflow ==> niveauWorkflow, conditionPassage, delaiMaxJours, idTypeDocument

2- Dépendances fonctionnelles issues des acteurs du système

- idUtilisateur ==> nomUtilisateur, prenomUtilisateur, emailUtilisateur, motDePasse, roleUtilisateur, serviceUtilisateur, gradeUtilisateur

3- Dépendances fonctionnelles issues de l'exécution des demandes

- idDemande ==> referenceDemande, dateCreationDemande, statutDemande, dateDerniereMajDemande, idTypeDocument, idUtilisateur
- idValeurChamp ==> valeurChampSaisie, idDemande, idChampFormulaire
- idPieceJointe ==> nomFichier, typeMime, tailleOctets, urlStockage, idDemande, idPieceRequise

4- Dépendances fonctionnelles issues du suivi, de la validation et de l'audit

- idEtapeValidation ==> niveauValidation, dateDebutValidation, dateFinValidation, decisionValidation, commentaireValidation, idDemande, idUtilisateur
- idHistoriqueAction ==> dateHistorique, acteurHistorique, actionHistorique, commentaireHistorique, idDemande
- idNotification ==> canalNotification, typeNotification, messageNotification, dateEnvoiNotification, statutEnvoiNotif, idUtilisateur, idDemande
- idJournalAudit ==> dateActionJournal, acteurJournal, actionJournal, detailsJournal

V- Structure d'Accès Théorique (SAT)

La Structure d'Accès Théorique (SAT) est une modélisation sous forme de graphe qui formalise les chemins d'accès optimaux entre les différentes clés de notre système. Elle permet de vérifier qu'à partir d'un identifiant racine, le système peut atteindre de manière déterministe l'ensemble des informations dépendantes.

Ce graphe met en évidence les relations de parenté (pivots) et préfigure la structure des futures associations (liens de 1 à N ou de N à N) du Modèle Conceptuel des Données

.

1- Liste des arcs de la SAT (Relations entre identifiants)

En nous basant sur le dictionnaire et les dépendances fonctionnelles, la dynamique des accès se structure à travers les liaisons d'identifiants suivantes :

- **idDemande** -> idTypeDocument *(Une demande fait référence à un unique type de document)*
- **idDemande** -> idUtilisateur *(Une demande est initiée par un unique utilisateur)*
- **idPieceRequise** -> idTypeDocument *(Une pièce requise est paramétrée pour un type de document)*
- **idChampFormulaire** -> idTypeDocument *(Un champ de formulaire appartient à un type de document)*
- **idEtapeWorkflow** -> idTypeDocument *(Une étape de workflow est configurée pour un type de document)*
- **idValeurChamp** -> idDemande, idChampFormulaire *(L'accès à une valeur saisie nécessite de connaître la demande et le champ concerné)*
- **idPieceJointe** -> idDemande, idPieceRequise *(L'accès à un fichier nécessite l'identifiant de la demande et de l'exigence documentaire)*
- **idEtapeValidation** -> idDemande, idUtilisateur *(Une étape de validation relie une demande à son validateur)*
- **idHistoriqueAction** -> idDemande *(L'historique des actions est directement indexé par la demande)*
- **idNotification** -> idUtilisateur, idDemande *(Une notification pointe vers un destinataire et un dossier)*

2- Représentation visuelle du Graphe d'Accès

Dans le manuscrit final, la SAT se représente sous la forme d'un schéma de réseaux. Les identifiants indépendants (comme idUtilisateur ou idTypeDocument) se placent au sommet de la hiérarchie graphique car ils servent de points d'entrée principaux. Les identifiants dépendants (comme idDemande, idPieceJointe) se situent en aval, reliés par les flèches d'accès.

Voici la cartographie logique de notre graphe :

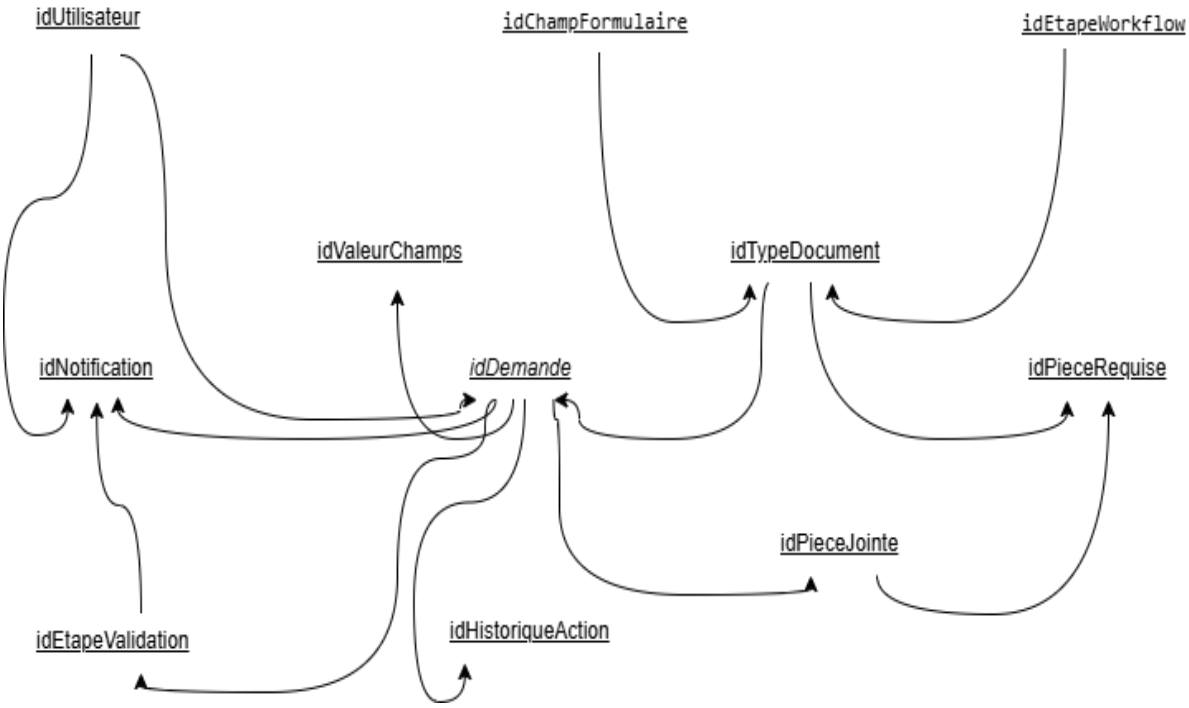


Figure 2 : Schéma de la Structure d'Accès Théorique (SAT)

3- Propriétés remarquables de la SAT du projet

1. **Le pivot central idDemande :** Le graphe démontre que la clé idDemande est le carrefour informationnel du système. Presque tous les accès transactionnels (historique, pièces jointes, validations) transitent par elle.
2. **Les référentiels stables :** idUtilisateur et idTypeDocument forment les tables de référence (ou dictionnaires applicatifs). Ils n'ont aucun antécédent dans le graphe d'accès (aucune flèche ne pointe *vers* eux), ce qui confirme leur autonomie.
3. **L'audit décorrélé :** L'identifiant idJournalAudit n'apparaît pas dans les liaisons directes de la SAT car il s'agit d'une structure transverse d'enregistrement linéaire en continu, n'intervenant pas dans le cycle de navigation fonctionnel d'un dossier.

VI- Modélisation des données

La modélisation des données permet de traduire le dictionnaire des données et les dépendances fonctionnelles en structures logiques, puis physiques, exploitables par un Système de Gestion de Base de Données (SGBD).

1- Modèle Conceptuel des Données (MCD)

Le Modèle Conceptuel des Données (MCD) représente la structure de l'information du système de manière sémantique, indépendamment de toute considération technique ou informatique. Il est composé **d'Entités** (regroupant les propriétés qui dépendent d'un identifiant unique) et **d'Associations** (matérialisant les liens sémantiques entre ces entités, qualifiées par des cardinalités).

- **Liste des Entités et de leurs attributs**

- **Utilisateur** : idUtilisateur (Identifiant), nomUtilisateur, prenomUtilisateur, emailUtilisateur, motDePasse, roleUtilisateur, serviceUtilisateur, gradeUtilisateur.
- **TypeDocument** : idTypeDocument (Identifiant), nomDocument, descriptionDocument, categorieDocument, serviceDocument, modelePdf.
- **PieceRequise** : idPieceRequise (Identifiant), nomPieceRequise, descriptionPiece, obligatoirePiece, formatAutorise, tailleMaxMo.
- **ChampFormulaire** : idChampFormulaire (Identifiant), nomChamp, typeChampFormulaire, obligatoireChamp, valeurDefaut, formatValidation, minChamp, maxChamp.
- **EtapeWorkflow** : idEtapeWorkflow (Identifiant), niveauWorkflow, conditionPassage, delaiMaxJours.
- **Demande** : idDemande (Identifiant), referenceDemande, dateCreationDemande, statutDemande, dateDerniereMajDemande.
- **ValeurChamp** : idValeurChamp (Identifiant), valeurChampSaisie.
- **PieceJointe** : idPieceJointe (Identifiant), nomFichier, typeMime, tailleOctets, urlStockage.

- **EtapeValidation** : idEtapeValidation (Identifiant), niveauValidation, dateDebutValidation, dateFinValidation, decisionValidation, commentaireValidation.
- **HistoriqueAction** : idHistoriqueAction (Identifiant), dateHistorique, acteurHistorique, actionHistorique, commentaireHistorique.
- **Notification** : idNotification (Identifiant), canalNotification, typeNotification, messageNotification, dateEnvoiNotification, statutEnvoiNotif.
- **JournalAudit** : idJournalAudit (Identifiant), dateActionJournal, acteurJournal, actionJournal, detailsJournal.

- Liste des Associations et Cardinalités

Les liaisons logiques et le dénombrement des participations (cardinalités minimales et maximales) se définissent ainsi :

- **Soumettre** (Entre *Utilisateur* et *Demande*) :
  - Un utilisateur peut soumettre de 0 à N demandes. (0,N)
  - Une demande est obligatoirement soumise par un et un seul utilisateur. (1,1)
- **Concerner** (Entre *TypeDocument* et *Demande*) :
  - Un type de document peut concerner 0 à N demandes. (0,N)
  - Une demande concerne obligatoirement un et un seul type de document. (1,1)
- **Définir Pièce** (Entre *TypeDocument* et *PieceRequise*) :
  - Un type de document définit 0 à N pièces requises. (0,N)
  - Une pièce requise est définie pour un et un seul type de document. (1,1)
- **Composer Formulaire** (Entre *TypeDocument* et *ChampFormulaire*) :
  - Un type de document compose son formulaire de 0 à N champs. (0,N)
  - Un champ de formulaire est lié à un et un seul type de document. (1,1)
- **Régir** (Entre *TypeDocument* et *EtapeWorkflow*) :
  - Un type de document est régi par 0 à N étapes de workflow. (0,N)

    - Une étape de workflow régit un et un seul type de document. (1,1)

- **Lier Saisie** (Entre *Demande* et *ValeurChamp*) :
    - Une demande possède 0 à N valeurs de champs saisies. (0,N)
    - Une valeur saisie est liée à une et une seule demande. (1,1)
- **Associer Champ** (Entre *ChampFormulaire* et *ValeurChamp*) :
    - Un champ de formulaire peut être associé à 0 à N valeurs saisies au fil des demandes. (0,N)
    - Une valeur saisie correspond à un et un seul champ de formulaire. (1,1)
- **Contenir Fichier** (Entre *Demande* et *PieceJointe*) :
    - Une demande contient 0 à N pièces jointes. (0,N)
    - Une pièce jointe appartient à une et une seule demande. (1,1)
- **Justifier Exigence** (Entre *PieceRequise* et *PieceJointe*) :
    - Une pièce requise est justifiée par 0 à N pièces jointes globales. (0,N)
    - Une pièce jointe justifie une et une seule exigence de pièce requise. (1,1)
- **Suivre Workflow** (Entre *Demande* et *EtapeValidation*) :
    - Une demande suit de 0 à N étapes de validation réelles. (0,N)
    - Une étape de validation concerne une et une seule demande. (1,1)
- **Traiter Validation** (Entre *Utilisateur* et *EtapeValidation*) :
    - Un utilisateur (ici un validateur) peut traiter 0 à N étapes de validation. (0,N)
    - Une étape de validation est traitée par un et un seul utilisateur. (1,1)
- **Tracer Action** (Entre *Demande* et *HistoriqueAction*) :
    - Une demande engendre 1 à N lignes d'historique d'actions. (1,N)
    - Une ligne d'historique concerne une et une seule demande. (1,1)
- **Notifier Destinataire** (Entre *Utilisateur* et *Notification*) :
    - Un utilisateur reçoit 0 à N notifications. (0,N)

- Une notification est adressée à un et un seul utilisateur. (1,1)

- **Notifier Demande** (Entre *Demande* et *Notification*) :
  - Une demande peut déclencher 0 à N notifications. (0,N)
  - Une notification est optionnellement ou obligatoirement liée à une demande. (0,1 ou 1,1 selon le besoin d'indépendance de l'alerte).

- MCD

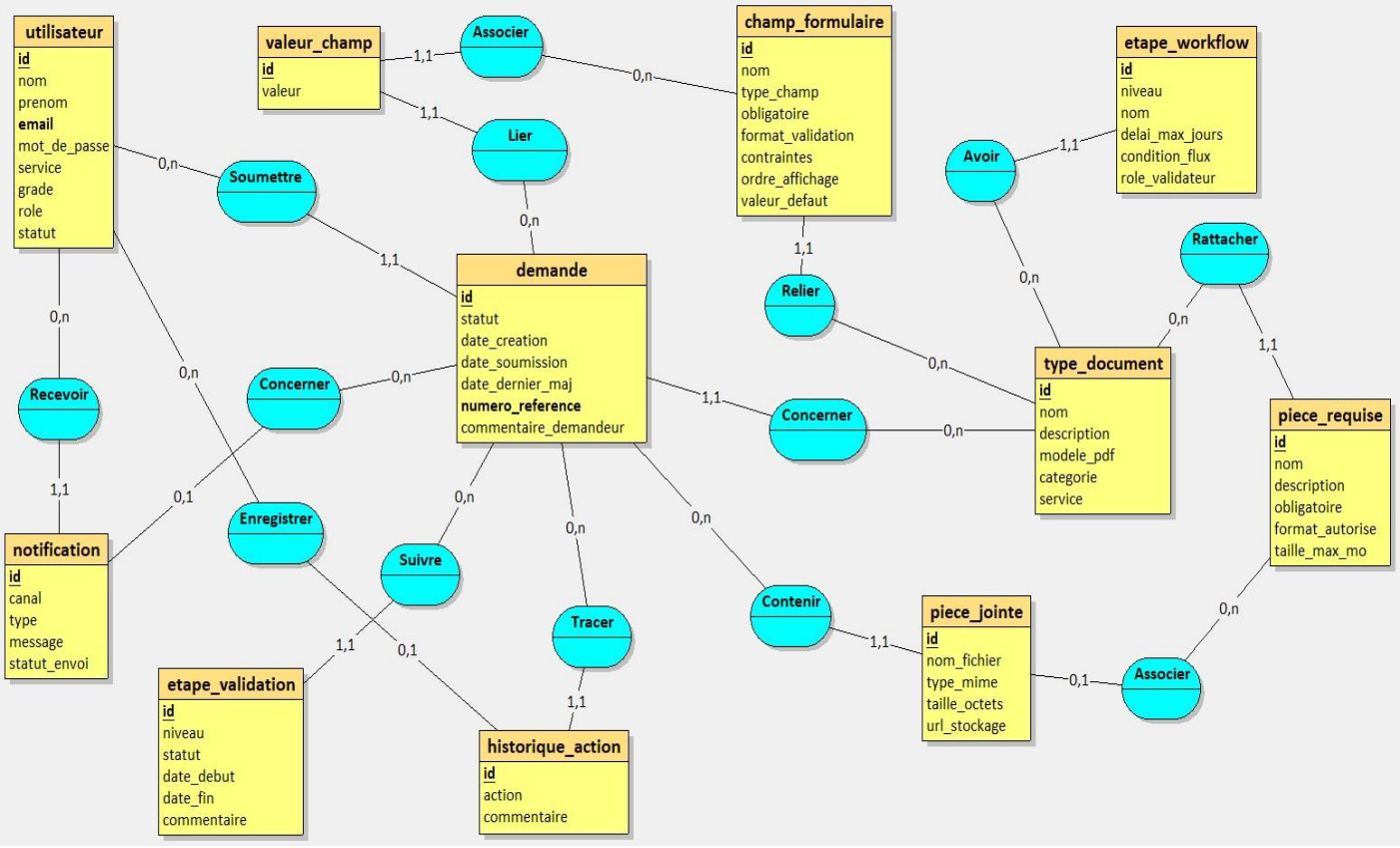


Figure 3 : Modèle Conceptuel de Données

L'analyse du Modèle Conceptuel des Données met en évidence une organisation claire et structurée du système d'information.

L'entité **Demande** constitue le cœur du modèle et centralise l'ensemble des interactions du système. Elle est associée :

- au **Type de document**, qui définit les règles, champs et pièces requis,

- à l'**Utilisateur**, auteur ou acteur du processus,
- aux **Étapes de validation**, permettant de gérer le workflow décisionnel,
- aux **Valeurs des champs** et **pièces jointes**, représentant les données saisies et les documents fournis,
- aux entités de **suivi et de traçabilité** telles que les notifications, l'historique des actions et le journal d'audit.

Les entités de paramétrage (**TypeDocument**, **ChampFormulaire**, **PieceRequise**, **EtapeWorkflow**) permettent une configuration dynamique du système sans modification structurelle de la base de données, ce qui renforce la flexibilité et la maintenabilité de la solution.

Enfin, le respect des cardinalités et des règles de gestion assure l'intégrité des données et reflète fidèlement le fonctionnement réel du processus métier. Ce Modèle Conceptuel des Données servira de fondement pour l'élaboration du Modèle Logique des Données (MLD) et du Modèle Physique des Données (MPD) présentés dans les sections suivantes.

2- Modèle Logique des Données Relationnelles (MLDR)

Le Modèle Logique des Données (MLD) est obtenu à partir du Modèle Conceptuel des Données (MCD) en appliquant les règles de transformation de la méthode MERISE.

Il consiste à traduire les entités, associations et cardinalités du MCD en structures relationnelles exploitables par un Système de Gestion de Base de Données Relationnelle (SGBDR).

Le MLD permet ainsi de définir :

- les tables relationnelles,
- les clés primaires,
- les clés étrangères,
- ainsi que les contraintes d'intégrité référentielle.

Comment réaliser un MLD ?

On représente ainsi les données issues de la modélisation Merise sous la forme suivante :

- Chaque ligne représente une table ;

- C’est toujours le nom de la table qui est écrit en premier ;
- Les champs sont listés entre parenthèses et séparés par des virgules ;
- Les clés primaires sont soulignées et placées au début de la liste des champs ;
- Les clés étrangères sont préfixées par un dièse.

Cette étape constitue un lien essentiel entre la modélisation conceptuelle et l’implémentation physique de la base de données.

- **Utilisateur** (idUtilisateur, nomUtilisateur, prenomUtilisateur, emailUtilisateur, motDePasse, roleUtilisateur, serviceUtilisateur, gradeUtilisateur)
- **TypeDocument** (idTypeDocument, nomDocument, descriptionDocument, categorieDocument, serviceDocument, modelePdf)
- **PieceRequise** (idPieceRequise, nomPieceRequise, descriptionPiece, obligatoirePiece, formatAutorise, tailleMaxMo, #idTypeDocument)
- **ChampFormulaire** (idChampFormulaire, nomChamp, typeChampFormulaire, obligatoireChamp, valeurDefaut, formatValidation, minChamp, maxChamp, #idTypeDocument)
- **EtapeWorkflow** (idEtapeWorkflow, niveauWorkflow, conditionPassage, delaiMaxJours, #idTypeDocument)
- **Demande** (idDemande, referenceDemande, dateCreationDemande, statutDemande, dateDerniereMajDemande, #idTypeDocument, #idUtilisateur)
- **ValeurChamp** (idValeurChamp, valeurChampSaisie, #idDemande, #idChampFormulaire)
- **PieceJointe** (idPieceJointe, nomFichier, typeMime, tailleOctets, urlStockage, #idDemande, #idPieceRequise)
- **EtapeValidation** (idEtapeValidation, niveauValidation, dateDebutValidation, dateFinValidation, decisionValidation, commentaireValidation, #idDemande, #idUtilisateur)
- **HistoriqueAction** (idHistoriqueAction, dateHistorique, acteurHistorique, actionHistorique, commentaireHistorique, #idDemande)
- **Notification** (idNotification, canalNotification, typeNotification, messageNotification, dateEnvoiNotification, statutEnvoiNotif, #idUtilisateur, #idDemande)
- **JournalAudit** (idJournalAudit, dateActionJournal, acteurJournal, actionJournal, detailsJournal)

Le MLD obtenu respecte les règles de normalisation, notamment la troisième forme normale (3FN), ce qui permet de limiter la redondance des données et d’assurer leur cohérence.

Les relations entre les tables sont assurées par des clés étrangères, garantissant l'intégrité référentielle entre les entités du système.

Ce modèle logique constitue la base de la conception du Modèle Physique des Données (MPD), qui permettra l'implémentation effective de la base de données dans un SGBDR.

3- Modèle Physique des Données (MPD)

a- Présentation du MPD

L'étape de création du MPD est presque une formalité comparée à la création du MCD. En s'appuyant sur des règles simples (et qui fonctionnent à tous les coups), l'analyste fait évoluer sa modélisation de haut niveau pour la transformer en un schéma plus proche des contraintes des logiciels de bases de données. Il s'agit de préparer l'implémentation dans un SGBDR.

Concrètement, cette étape permet de **construire la structure finale de la base de données** avec les différents liens entre les éléments qui la composent. Pour la peine, on change aussi de vocabulaire :

- Les entités se transforment en tables ;
- Les propriétés se transforment en champs (ou attributs) ;
- Les propriétés se trouvant au milieu d'une relation génèrent une nouvelle table ou glissent vers la table adéquate en fonction des cardinalités de la relation ;
- Les identifiants se transforment en clés et se retrouvent soulignés. Chaque table dispose d'au minimum 1 clé dite primaire ;
- Les relations et les cardinalités se transforment en champs parfois soulignés : il s'agit de créer des « clés étrangères » reliées à une « clé primaire » dans une autre table.

b- Description des tables du MPD

Le Modèle Physique des Données (MPD) représente la traduction concrète du Modèle Logique des Données dans un système de gestion de base de données relationnel.

Il définit précisément les tables, les clés primaires, les clés étrangères, les types de données ainsi que les contraintes d'intégrité nécessaires à l'implémentation effective de la base de données.

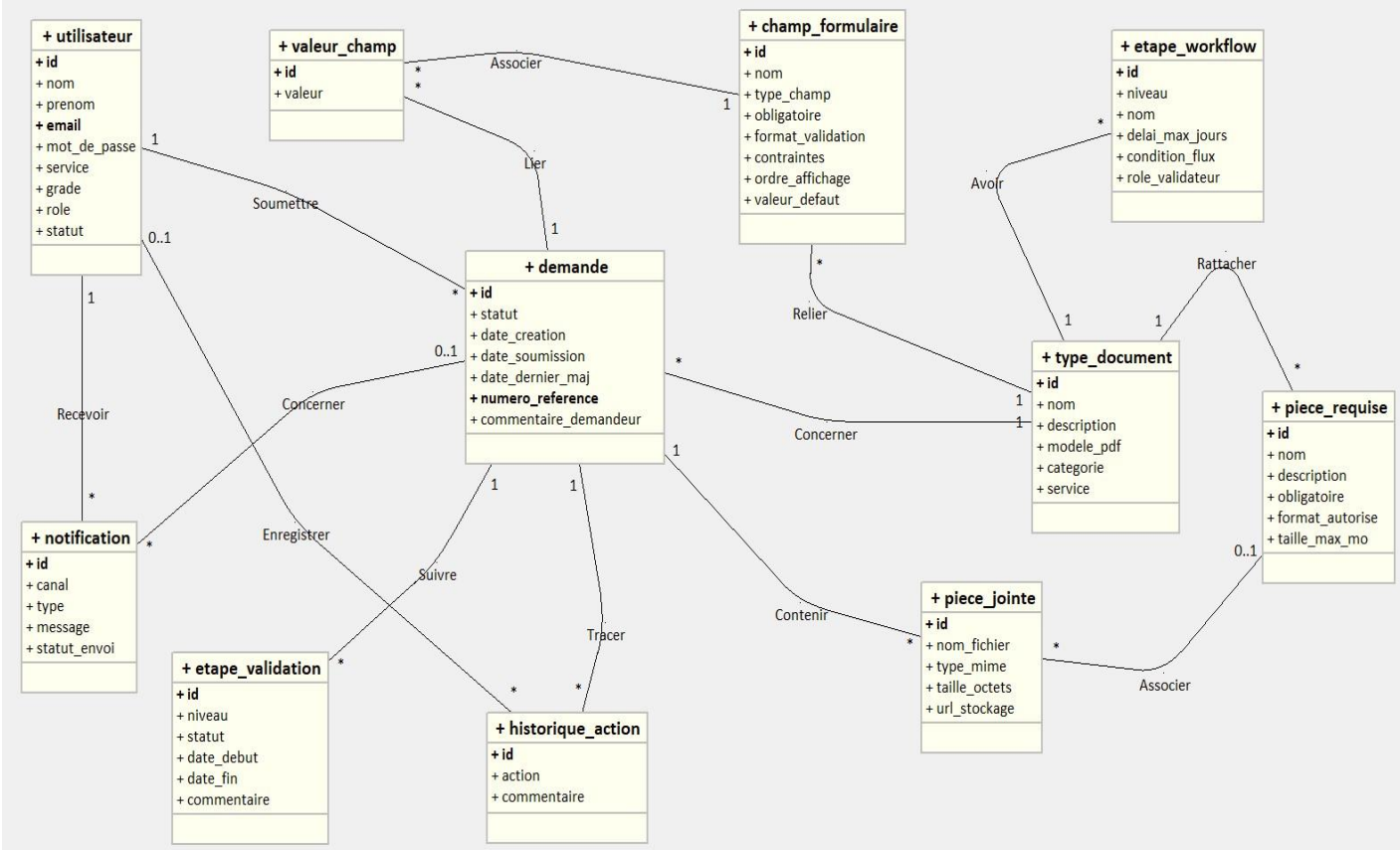


Figure 4 : Schéma du Modèle Physique des Données (MPD)

Le MPD présenté a été conçu pour garantir la cohérence, la sécurité et la performance du stockage des données, tout en respectant les règles de gestion définies lors de la modélisation conceptuelle.

Cette structure permet de gérer efficacement les demandes documentaires, les utilisateurs, les workflows de validation, les pièces jointes ainsi que les mécanismes de traçabilité et d'audit.

Après avoir défini l'ensemble des modèles de données nécessaires à la conception du système d'information, la suite de ce travail est consacrée à la réalisation de la modélisation des traitements.

VII- Modélisation des traitements

1- Modèle de Conceptuel de Traitement (MCT)

Dans la méthodologie Merise, il y a tout un pan de l'analyse qui porte sur les **traitements**. Le **MCT** ou **Modèle Conceptuel des Traitements** en fait partie. Il présente graphiquement, sous la forme de schémas avec des éléments bien définis, **les traitements qui doivent être informatisés**. Il permet d'identifier le fonctionnement du système d'information. Dit autrement, il répond au QUOI.

Le MCT est le miroir du MCD. Le MCD se focalise sur les données alors que le MCT se focalise sur les réponses à donner lorsqu'un événement survient. Les 2 sont complémentaires et agissent au niveau conceptuel, c'est à dire qu'il cherchent à modéliser le fonctionnement d'un point de vue métier.

Le MCT détaille les processus en événements et en opérations :

- un **événement** est un déclencheur ou un résultat. Il peut être externe ou interne (dans le Système d'Information) ;
- la **synchronisation** est l'ensemble des conditions à remplir pour déclencher une opération en fonction des événements entrants. On utilise ici la logique booléenne ;
- une **opération** est effectuée suite au déclenchement d'un ou plusieurs événements. C'est la somme des actions à réaliser d'un coup. On utilise un verbe pour l'exprimer ;
- des **règles d'émission** sont définis dans l'opération
- un **processus** est une suite chaînée d'événements et d'opérations. Dans sa version la plus simple, un MCT à un événement en entrée, une opération et un événement en sortie.

Afin d'illustrer de manière concrète le fonctionnement du système d'information étudié, nous présentons ci-après les différents Modèles Conceptuels de Traitements (MCT) relatifs aux principaux processus métiers identifiés. Ces modèles permettent de décrire, pour chaque événement métier, les traitements déclenchés ainsi que les résultats attendus.

1.1- MCT 1 : Soumission d'une demande

Ce processus gère la transition d'un document du statut de brouillon vers son entrée officielle dans le système après vérification de sa conformité réglementaire.

A- Schéma conceptuel

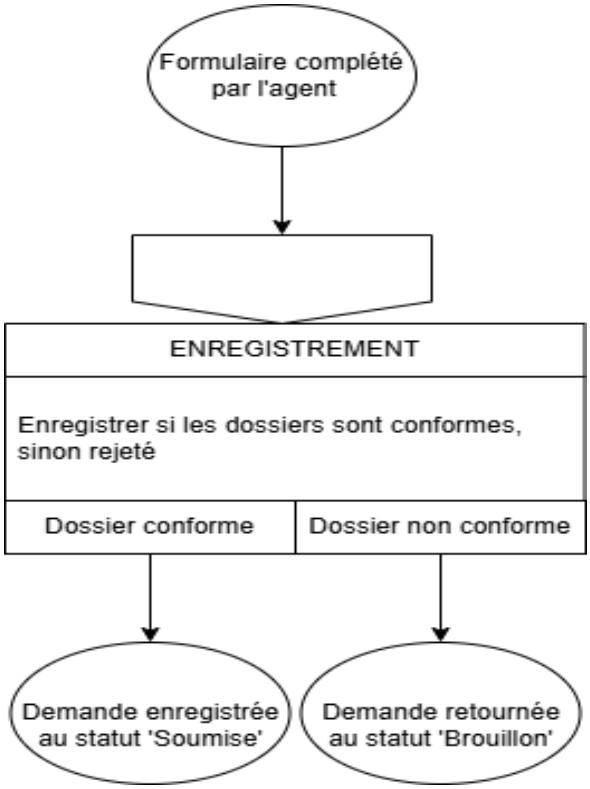


Figure 5 : MCT 1 : Soumission d'une demande)

B- Spécification des éléments du traitement

- **Événement déclencheur :** *Formulaire complété par l'agent* (L'utilisateur clique sur le bouton de soumission).
- **Synchronisation :** Toujours (L'action de vérification démarre dès la soumission).
- **Règles de gestion de l'opération OP01 :**

1. Vérification de la présence de l'ensemble des pièces requises obligatoires.
2. Validation du format des données saisies (contrôle par expressions régulières).
3. Attribution d'un numéro de référence unique et immuable.

- **Règles d'émission et Résultats :**
  - Si [Dossier conforme] : Émission de l'événement *Demande enregistrée au statut 'Soumise'*.

- Si [Dossier non conforme] : Émission de l'événement *Demande retournée au statut 'Brouillon'* avec rapport d'erreurs.

1.2- MCT 2 : Instruction de la demande (Workflow)

Ce processus pilote le cœur du réacteur : l'évaluation d'un dossier par une autorité compétente pour le niveau de validation en cours.

A-Schéma conceptuel

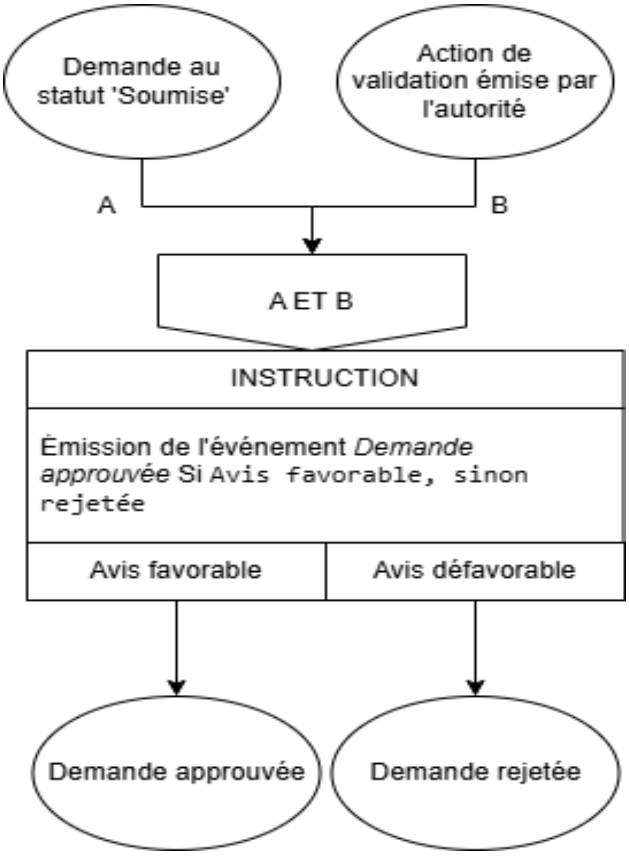


Figure 6 : MCT 2 : Instruction de la demande (Workflow)

B- Spécification des éléments du traitement

- **Événements déclencheurs :**

  1. *Demande au statut 'Soumise'* (Dossier disponible dans la file).
  2. *Action de validation émise par l'autorité* (Le validateur valide ou rejette depuis son tableau de bord).

- **Synchronisation :** ET (L'opération ne peut s'exécuter que si le dossier existe **ET** que le validateur a soumis sa décision).
- **Règles de gestion de l'opération OP02 :**

  1. Analyse de la décision saisie par le validateur.
  2. Enregistrement des commentaires d'instruction.

- **Règles d'émission et Résultats :**

  - Si [Avis favorable] : Émission de l'événement *Demande approuvée* (qui déclenchera le passage au niveau suivant ou la clôture).
  - Si [Avis défavorable] : Émission de l'événement *Demande rejetée* (fin définitive du circuit).

1.3- MCT 3 : Production du document

Ce processus automatisé s'enclenche immédiatement après l'approbation finale pour matérialiser le document administratif officiel.

A-Schéma conceptuel

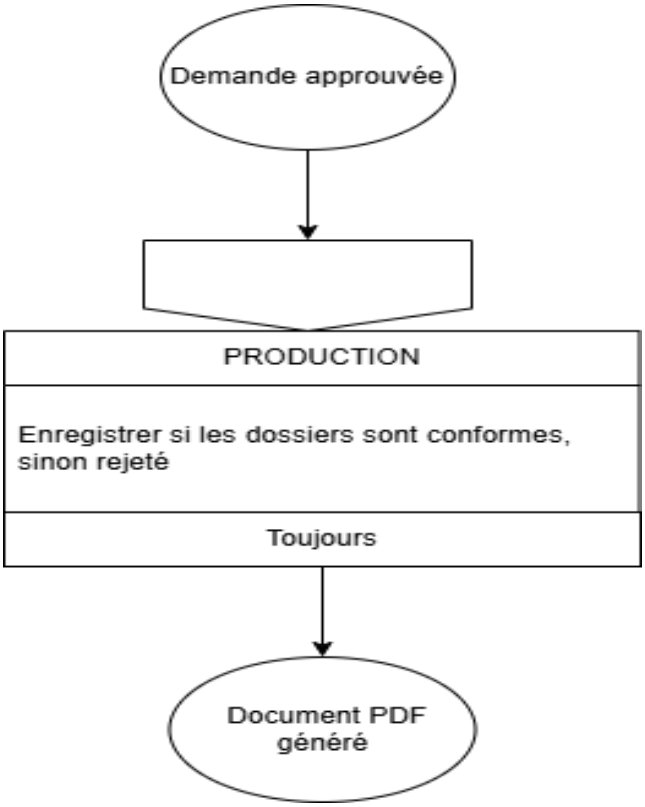


Figure 7 : MCT 3 : Production du document

B- Spécification des éléments du traitement

- **Événement déclencheur :** *Demande approuvée* (Le dernier niveau hiérarchique a donné son accord).
- **Synchronisation :** Toujours (Dès l'approbation finale obtenue, l'édition commence).
- **Règles de gestion de l'opération OP03 :**

1. Extraction des métadonnées du formulaire stockées en base de données.
2. Injection dynamique des données dans le modèle PDF (modele_pdf).
3. Scellement numérique du document et sauvegarde sur le serveur de stockage.

- **Règles d'émission et Résultats :**

  - [Toujours] : Émission de l'événement *Document PDF généré* avec mise à jour du statut global à 'Clôturé'.

1.4- MCT 4 : Traçabilité et Notification

Ce processus gère la transition d'un document du statut de brouillon vers son entrée officielle dans le système après vérification de sa conformité réglementaire.

A- Schéma conceptuel

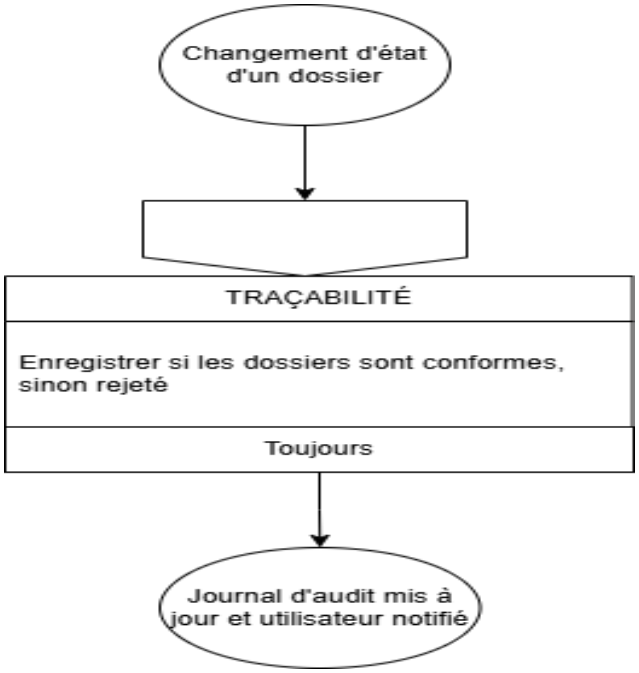


Figure 8 : MCT 4 : Traçabilité et Notification

B- Spécification des éléments du traitement

- **Événement déclencheur :** *Changement d'état d'un dossier* (Une demande est créée, soumise, validée ou rejetée).
- **Synchronisation :** Toujours (Toute modification structurelle doit être immédiatement tracée).
- **Règles de gestion de l'opération OP04 :**

1. Journalisation de l'ancienne et de la nouvelle valeur de l'entité modifiée dans journal_audit.
2. Enregistrement de l'action dans la table historique_action.
3. Détermination du canal de communication optimal (Email/SMS) selon le profil de l'utilisateur.
4. Génération et envoi du message d'alerte.

- **Règles d'émission et Résultats :**

- [Toujours] : Émission de l'événement *Journal d'audit mis à jour et utilisateur notifié*.

En définissant précisément les règles métier, les événements déclencheurs et les résultats attendus, le Modèle Conceptuel des Traitements (MCT) a permis de figer le comportement logique et le « **QUOI** » du système d'information. Afin de compléter cette analyse conceptuelle, il convient désormais d'introduire les notions de temps, de lieux et de profils d'acteurs à travers le Modèle Organisationnel des Traitements (MOT) pour définir précisément le « **QUI**, **QUAND** et **OÙ** » de chaque opération.

VIII- Modélisation Organisationnelle des Traitements (MOT)

1- Fondements théoriques et Objectifs du MOT

Si le Modèle Conceptuel des Traitements (MCT) a permis de définir **« QUOI »** faire sans se soucier des contraintes de l'organisation, le **Modèle Organisationnel des Traitements (MOT)** apporte une dimension opérationnelle en répondant aux questions fondamentales suivantes :

- **QUI** exécute la tâche ? (Rôles, acteurs humains ou processus système automatisés) ;
- **QUAND** la tâche est-elle exécutée ? (Délais, fréquences, événements temporels) ;
- **OÙ** et **COMMENT** s'exécute-t-elle ? (Poste de travail, nature du traitement : interactif, batch, ou automatique).

Le passage du MCT au MOT se traduit par l'éclatement ou le regroupement des opérations conceptuelles en **Procédures Organisationnelles (PO)**. Une PO est un ensemble de traitements exécutés de manière continue par un même poste de travail (acteur ou machine) utilisant des ressources spécifiques.

2- Typologie des concepts organisationnels retenus

Pour notre système de gestion de documents, nous classons les éléments selon trois critères stricts :

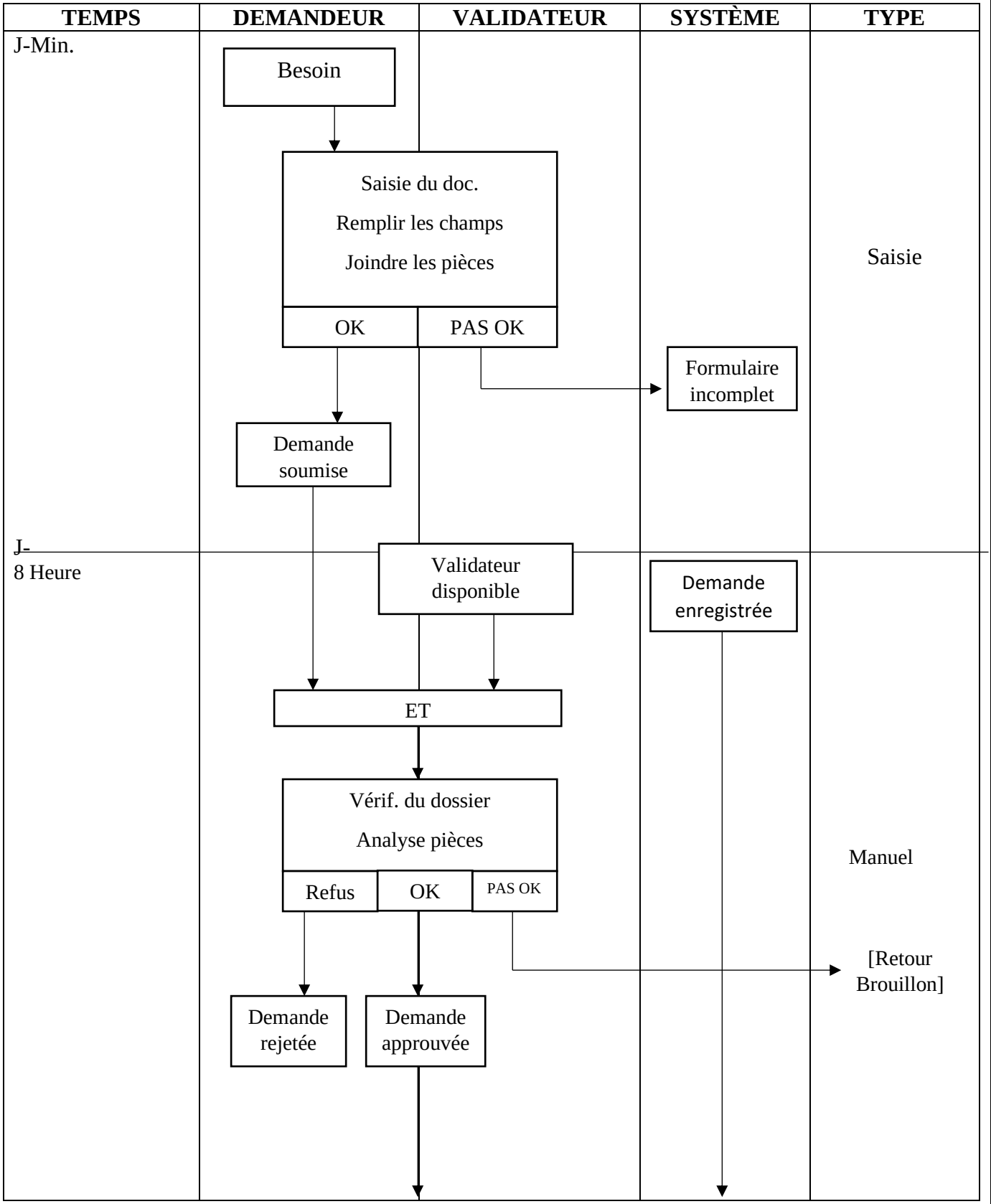
1. **Les Postes de Travail (Acteurs) :**
    - **Agent (Demandeur) :** Acteur humain, externe au cœur du SI, initiateur de la demande.
    - **Validateur (Autorité/RH/Manager) :** Acteur humain interne, disposant des droits d'instruction.
    - **Système (Moteur applicatif) :** Acteur logiciel automatisé (serveur de backend, scripts asynchrones).
2. **La Nature des Traitements :**
    - **TR (Temps Réel / Interactif) :** Dialogue direct entre l'homme et la machine via l'interface web.
    - **Auto (Automatique) :** Exécution asynchrone transparente par le serveur, déclenchée par un signal système.
3. **Le Temps et la Périodicité :**
    - **Au fil de l'eau :** Traitement unitaire déclenché immédiatement dès la survenue de l'événement.

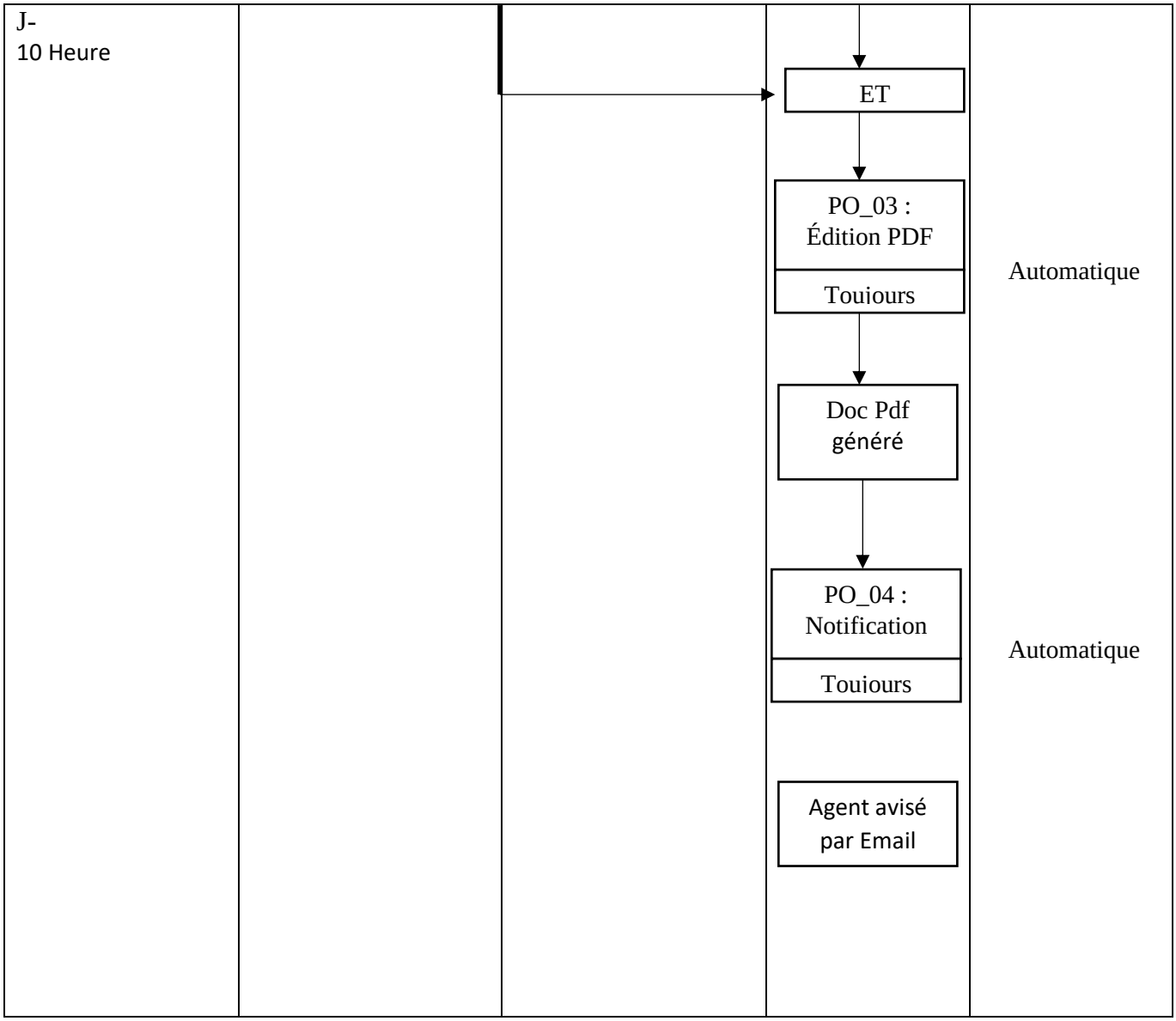
3- Tableau Synoptique Global du MOT

Code PO	Intitulé de la Procédure Organisationnelle	Poste de Travail (Acteur)	Nature (Type)	Périodicité	Description détaillée de la procédure
PO_01	Soumission et validation de formulaire	Agent demandeur / Système	Interactif (TR)	Au fil de l'eau	L'agent saisit les informations via le portail web. Lors de la soumission, le système contrôle la validité des données, vérifie la présence des pièces jointes obligatoires et enregistre la demande si tout est conforme.

PO_02	Instruction et arbitrage du dossier	Validateur (autorité administrative)	Interactif (TR)	Selon planning (max 48h)	Le validateur reçoit une notification dans son tableau de bord, consulte le dossier complet, analyse les pièces justificatives et enregistre sa décision (validation ou rejet) accompagnée de commentaires.
PO_03	Édition documentaire et signature	Système (moteur de génération PDF)	Automatique	Instantané	Dès validation finale, le système récupère les données de la demande, génère automatiquement le document officiel au format PDF et applique une signature ou un scellement numérique pour garantir son intégrité.
PO_04	Traçabilité globale et notifications	Système (worker asynchrone)	Automatique	Continu (24h/24 – 7j/7)	Le système enregistre toutes les actions dans un journal d'audit, conserve l'historique des modifications et déclenche l'envoi de notifications (email/SMS) aux utilisateurs concernés en cas de changement de statut.

4- Schémas Réels et Détaillés des Procédures (Représentation MOT)


TEMPS
DEMANDEUR
VALIDATEUR
SYSTÈME
TYPE
J-Min.
Besoin
Saisie du doc.
Remplir les champs
Joindre les pièces
OK
PAS OK
Formulaire incomplet
Saisie
Demande soumise
J-
8 Heure
Validateur disponible
Demande enregistrée
ET
Vérif. du dossier
Analyse pièces
Refus
OK
PAS OK
Manuel
[Retour Brouillon]
Demande rejetée
Demande approuvée



- **Phase J-1 Minute (Saisie) :** La procédure est initiée en mode **Interactif / Saisie** par l'Agent. L'opération intègre un contrôle de surface. Si l'un des formats ou fichiers requis manque (Pas OK), le système émet un résultat bloquant (Formulaire incomplet). Si tout est conforme (OK), l'événement Demande soumise est instancié.
- **Phase J / 8 Heures (Manuel) :** C'est une phase **Manuelle** de synchronisation logique. L'instruction exige la présence conjointe (ET) de la notification de la demande en base de données et de la disponibilité physique de l'autorité de validation. L'opération émet trois résultats exclusifs : le refus définitif, le retour à l'envoyeur pour correction, ou l'approbation.
- **Phase J / 10 Heures (Automatique) :** Dès l'émission de l'événement Demande approuvée, le système prend le relais de manière **Automatique**. La PO_03 compile les données du formulaire

avec le template pour stocker le fichier physique, tandis que la PO_04 s'occupe de la traçabilité asynchrone et de l'envoi du mail de notification à l'agent.

Le Modèle Organisationnel des Traitements (MOT) a permis de structurer la répartition des tâches entre l'homme et la machine, d'attribuer les responsabilités aux différents acteurs du système et de définir les modalités d'exécution temporelles. L'ensemble du comportement logique (MCT) et organisationnel (MOT) étant désormais parfaitement défini, il est possible de quitter la phase d'analyse conceptuelle pour aborder la conception technique du logiciel à travers l'architecture applicative et la structure des composants.

PARTIE 3 : REALISATION ET DISCUSSION

Chapitre 6 : REALISATION DE L'APPLICATION

I- Environnement et outils de développement

La phase de réalisation de l'application a nécessité la mise en place d'un environnement de développement à la fois moderne, stable et largement adopté dans le milieu professionnel du développement web. Ce choix s'est porté sur des technologies reconnues pour leur robustesse, leur communauté active et leur adéquation avec les exigences d'applications web actuelles à vocation professionnelle.

1- Environnement matériel et système

L'environnement technique global repose sur les choix suivants :

- **Système d'exploitation** : Windows 10/11 Professionnel Ce système a été privilégié pour sa compatibilité optimale avec l'ensemble des outils de développement utilisés (notamment Visual Studio Code, les JDK récents, les outils Angular CLI et Spring Initializr), ainsi que pour sa facilité d'installation des dépendances et des pilotes nécessaires.
- **Architecture globale du système** : Architecture Client–Serveur L'application adopte une séparation claire entre la partie cliente (navigateur web) et la partie serveur (API + base de données), ce qui correspond au modèle dominant dans le développement d'applications web modernes.
- **Type d'application** : Application web monopage (Single Page Application – SPA) avec une API REST et un frontend découplé Ce choix d'architecture permet d'obtenir plusieurs avantages significatifs : une meilleure évolutivité future (possibilité d'ajouter une application mobile ou une version tablette sans refonte majeure du backend), une maintenance simplifiée grâce à la séparation des responsabilités, des performances accrues grâce au chargement asynchrone des données, et enfin une expérience utilisateur plus fluide et réactive comparable aux applications natives.

Ce choix permet une meilleure évolutivité, une maintenance simplifiée et une séparation claire des responsabilités entre les différentes couches du système.

2- Outils de développement

Plusieurs outils ont été mobilisés durant les différentes phases du projet, chacun étant choisi pour ses fonctionnalités spécifiques et sa popularité dans l'écosystème professionnel :

- **Visual Studio Code (VS Code)** Éditeur de code principal utilisé aussi bien pour le développement backend que frontend. Sa légèreté, sa rapidité et surtout son écosystème très riche d'extensions (Angular Language Service, Spring Boot Extension Pack, Prettier, ESLint, GitLens, Java Extension Pack, Docker, etc.) en font un outil extrêmement productif. La synchronisation des paramètres et des extensions via le compte GitHub a également permis de conserver un environnement homogène sur plusieurs postes de travail.
- **GitLab** Plateforme utilisée pour la gestion centralisée du code source, le versionnement, le suivi des tâches et la collaboration. Un dépôt privé a été créé avec une organisation claire des branches (main, develop, feature/, *hotfix/*) suivant le **Gitflow**. Les fonctionnalités de **Merge Requests**, de revue de code et de **CI/CD** (via GitLab CI) ont été exploitées pour automatiser les tests et les déploiements.
- **Outils de modélisation de données** :
  - **dbdiagram.io** : outil en ligne intuitif et collaboratif permettant de réaliser rapidement des **MCD** (Modèle Conceptuel de Données) et **MLD** (Modèle Logique de Données) grâce à son langage textuel simple et visuel.
  - **JMerise** : utilisé en complément pour les modélisations plus classiques et la génération automatique du **MPD** (Modèle Physique de Données) compatible avec le SGBD retenu. Ces deux outils ont permis de passer très efficacement de la conception à l'implémentation de la base de données.

II- Architecture technique de l'application

1- Architecture générale

L'application a été conçue selon une **architecture moderne découplée** composée de plusieurs couches distinctes et faiblement couplées :

- **Frontend web** : développé avec le framework **Angular** (version 17 ou supérieure selon la date de réalisation)
- **Backend** : API RESTful développée avec **Spring Boot** (version 3.x)
- **Base de données** : relationnelle (MySQL / PostgreSQL selon le contexte du projet)
- **Couche de sécurité** : implémentation de mécanismes d'authentification et d'autorisation (JWT, Spring Security)
- **Services transversaux** : validation des données, gestion des exceptions, logging, documentation API (OpenAPI/Swagger)

Cette architecture en couches présente de nombreux avantages concrets :

- **Séparation claire des responsabilités** : le frontend se concentre exclusivement sur l'interface et l'expérience utilisateur, tandis que le backend gère la logique métier, les règles de sécurité et l'accès aux données.
- **Faible couplage** entre frontend et backend : ils communiquent uniquement via une API REST bien documentée, ce qui facilite les évolutions indépendantes des deux parties.
- **Évolutivité accrue** : possibilité d'ajouter très facilement de nouvelles interfaces clientes (application mobile avec Ionic/React Native, application desktop avec Electron, etc.).
- **Meilleure maintenabilité** : chaque couche peut être testée, déboguée et déployée de manière autonome.
- **Sécurité renforcée** : les mécanismes d'authentification et de protection des endpoints sont centralisés dans le backend.
- **Facilité de déploiement** : le frontend peut être servi de manière statique (via Nginx, Netlify, Vercel, etc.), tandis que le backend peut être conteneurisé avec Docker et orchestré via Kubernetes ou simplement déployé sur un serveur d'application.

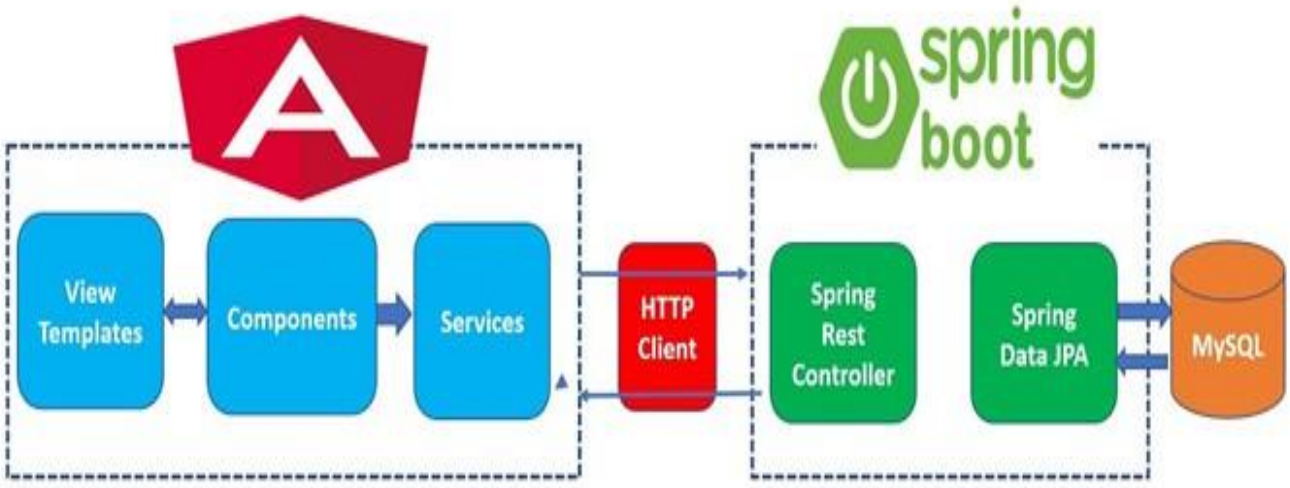


Figure 9 : Schéma de l'architecture globale du projet

2- Architecture backend

Le backend de l'application a été entièrement développé en **Java** avec le framework **Spring Boot**, qui constitue aujourd'hui l'un des choix les plus robustes, matures et largement adoptés dans l'écosystème Java pour la création d'API REST modernes.

L'organisation du code suit scrupuleusement le pattern **architecture hexagonale** (ou architecture en couches propre) combinée aux bonnes pratiques recommandées par la communauté Spring :

- **Couche Controller (ou couche de présentation/API)** Cette couche est responsable de la réception des requêtes HTTP entrantes, de la validation initiale des données reçues (via @Valid et Bean Validation), du mapping vers les objets métier et du renvoi des réponses HTTP (statut, corps, headers). Les contrôleurs sont organisés par domaine fonctionnel (ex. : UserController, DocumentController, NotificationController) et utilisent les annotations Spring classiques (@RestController, @RequestMapping, @GetMapping, @PostMapping, etc.).

- **Couche Service (logique métier)** C'est le cœur de l'application. Cette couche contient l'ensemble des règles métier, les traitements complexes, les orchestrations entre plusieurs entités, les validations métier poussées, ainsi que la gestion des cas d'erreur spécifiques au domaine. Les

services sont conçus pour être **stateless**, **testables** (unitaires et d'intégration) et **réutilisables**. Ils sont annotés @Service et injectés via l'inversion de contrôle de Spring. Une séparation claire a été maintenue entre les **services métier** (logique pure) et les **services utilitaires** (gestion des fichiers, génération de PDF, envoi d'emails, etc.).

- **Couche Repository (accès aux données)** Cette couche est implémentée grâce à **Spring Data JPA** couplé à **Hibernate** comme implémentation par défaut du fournisseur JPA. Pour chaque entité métier importante, une interface Repository a été créée en étendant **JpaRepository** ou **CrudRepository**, ce qui permet de bénéficier automatiquement de nombreuses méthodes CRUD sans écrire de code. Lorsque la requête devient complexe, des méthodes personnalisées ont été définies avec **@Query** (JPQL ou SQL natif) ou via l'utilisation des **Specifications** et **Criteria API** pour les recherches dynamiques et filtrées.

- **Entités (Entities)** Les classes annotées @Entity représentent la modélisation objet des tables de la base de données. Elles utilisent les annotations JPA classiques (@Id, @GeneratedValue, @Column, @ManyToOne, @OneToMany, @ManyToMany, etc.) ainsi que les annotations de validation (@NotNull, @Size, @Email, etc.). Une attention particulière a été portée aux relations bidirectionnelles (gestion correcte des cascades, orphanRemoval, mappedBy) pour éviter les problèmes classiques de LazyInitializationException ou de boucles infinies lors de la sérialisation JSON.

- **DTO (Data Transfer Objects)** Afin de découpler complètement la structure interne des entités de celle exposée à l'extérieur, un ensemble de **DTO** a été créé pour chaque opération significative (Create, Update, Response, List, Detailed, etc.). Le mapping entre entités et DTO est effectué soit manuellement (via des constructeurs ou des méthodes statiques), soit à l'aide de **MapStruct** (générateur de code très performant et type-safe), ce qui permet de limiter drastiquement les risques de fuites de données sensibles et d'éviter les problèmes de surcharge (over-fetching/under-fetching).

Cette organisation en couches bien définies facilite grandement :

- La maintenance à long terme
- Les tests unitaires et d'intégration
- La scalabilité horizontale
- La compréhension du code par les nouveaux développeurs

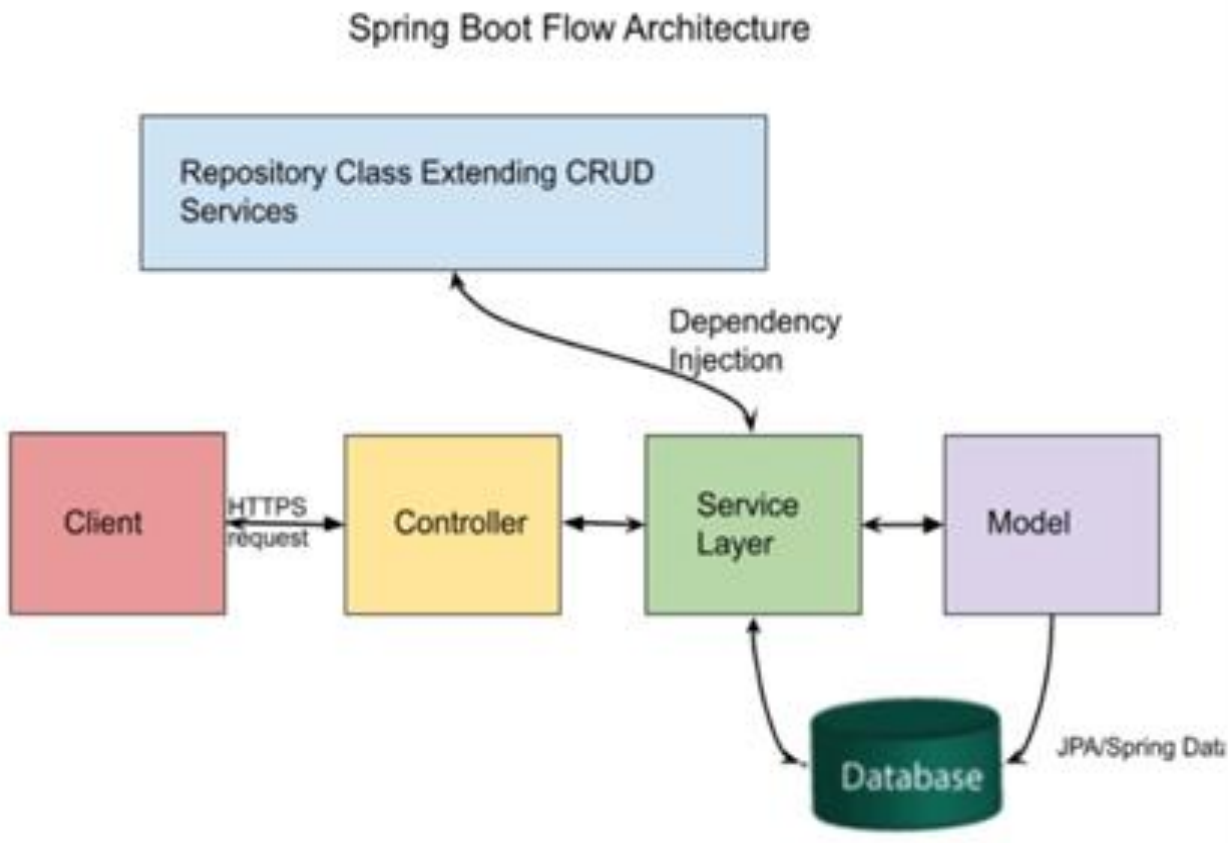


Figure 10 : Architecture Backend

L'API expose des endpoints REST respectant les standards HTTP (GET, POST, PUT, DELETE).

3- Architecture frontend

Le frontend a été développé avec le framework **Angular** (version 17 ou supérieure), en adoptant une architecture modulaire, scalable et conforme aux bonnes pratiques actuelles de l'écosystème Angular.

Les principaux éléments structurants sont les suivants :

- **Organisation modulaire** L'application est découpée en **modules fonctionnels** (Feature Modules) chargés à la demande (lazy-loading) :
    - AuthModule (connexion, inscription, récupération de mot de passe)
    - DashboardModule
    - UserManagementModule
    - DocumentModule
    - SettingsModule etc. Cette approche réduit considérablement le temps de chargement initial et optimise les performances globales de l'application.
- **Composants (Components)** Les composants constituent les briques de base de l'interface utilisateur. On distingue principalement :
    - **Smart components** (ou containers) : responsables de la communication avec les services, de la gestion de l'état et du passage des données aux composants de présentation
    - **Dumb components** (ou presentational) : purement graphiques, reçoivent les données via @Input et émettent des événements via @Output Cette séparation favorise la réutilisabilité et facilite les tests.
- **Services** Les services Angular (injectables) centralisent la logique de communication avec le backend via **HttpClient**. On retrouve notamment :
    - AuthService (gestion de l'authentification, stockage du token)
    - ApiService (générique pour les appels CRUD)
    - UserService, DocumentService, NotificationService, etc. Tous les services sont fournis au niveau du root ou du module concerné (strategy providedIn: 'root' ou scoped).
- **Guards** Plusieurs types de guards ont été implémentés pour sécuriser la navigation :
    - **AuthGuard** : bloque l'accès aux routes protégées si l'utilisateur n'est pas authentifié
    - **RoleGuard** : restreint l'accès selon les rôles/permissions de l'utilisateur (ex. : ADMIN, MANAGER, USER)
    - **NoAuthGuard** : empêche l'accès aux pages de login/inscription quand l'utilisateur est déjà connecté
- **Interceptors** Un **HttpInterceptor** principal a été mis en place pour :
    - Ajouter automatiquement le token JWT dans l'en-tête Authorization sur tous les appels authentifiés
    - Gérer les erreurs globales (401 → redirection vers login, 403 → message d'accès refusé, 500 → page d'erreur générique)
    - Ajouter des headers personnalisés (Accept-Language, timezone, etc.) si nécessaire
- **Autres éléments importants**

- **State management** : utilisation de **Services simples** + **BehaviorSubject** pour les besoins courants, avec possibilité d'évolution vers **NgRx** ou **Akita** si la complexité augmente
- **Gestion des formulaires** : utilisation systématique des **Reactive Forms** (plus puissants et testables)
- **Internationalisation (i18n)** : préparation de l'application à la multi-langue via @angular/localize
- **Thématisation** : utilisation de **SCSS** avec variables globales et composants stylisés selon la charte graphique du projet

Cette architecture frontend, bien structurée et respectueuse des principes de **Single Responsibility** et **Component-Based Design**, permet d'obtenir une application maintenable, évolutive et offrant une excellente expérience développeur.

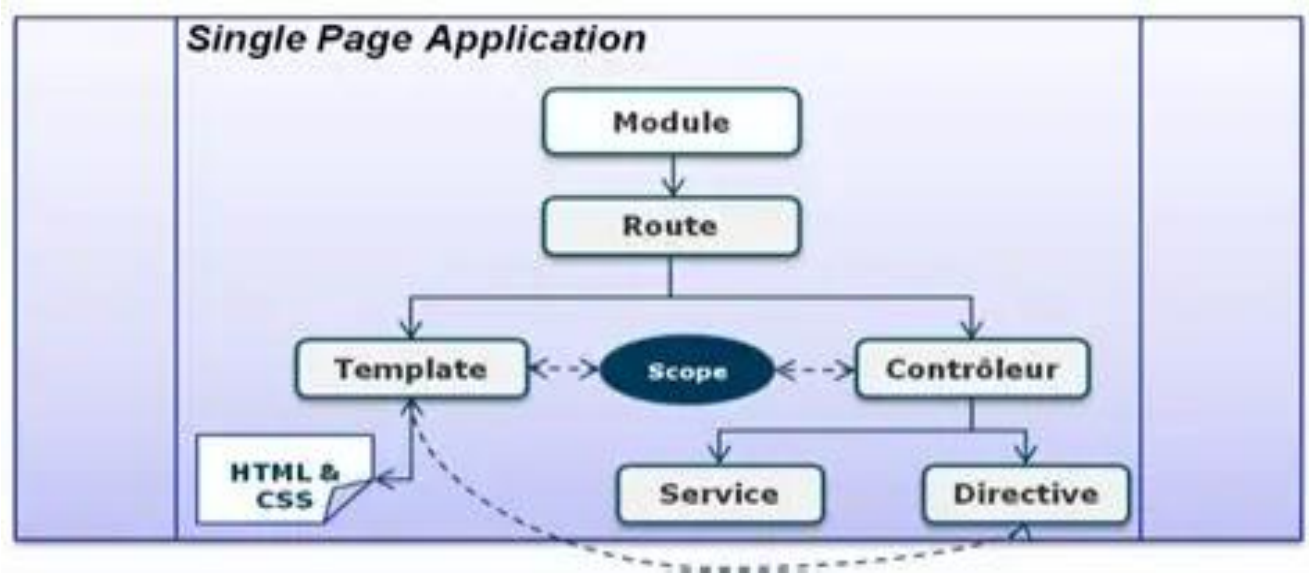


Figure11 : Architecture Frontend

Angular permet une expérience utilisateur dynamique et réactive.

III- Implémentation des fonctionnalités principales

1- Authentification et gestion des utilisateurs

L'application intègre un système d'authentification moderne et hautement sécurisé, basé sur le standard **JWT** (JSON Web Token) combiné à **Spring Security 6+**, qui reste la référence en 2026 pour les applications REST découplées.

Les principales caractéristiques mises en œuvre sont les suivantes :

- **Authentification stateless basée sur JWT** : Lors de la connexion, l'utilisateur fournit ses identifiants (email + mot de passe). Le backend vérifie les credentials contre la base de données (mot de passe hashé avec **BCrypt**), puis génère un **access token** JWT (durée de vie courte : 15-30 minutes) et, optionnellement, un **refresh token** (durée plus longue : 7 jours) stocké de manière sécurisée côté client (HttpOnly cookie ou localStorage avec protections XSS/CSRF). Cette approche stateless permet une scalabilité horizontale sans session serveur.

- **Gestion fine des rôles et permissions** : Trois rôles principaux ont été définis :
    - **Demandeur** : peut créer, modifier (en brouillon) et soumettre des demandes
    - **Validateur** : peut consulter, approuver, rejeter ou demander des corrections sur les demandes qui lui sont assignées
    - **Administrateur** : accès total (gestion des utilisateurs, configuration des workflows, consultation des audits) Les autorisations sont vérifiées à deux niveaux :
    - Backend : via **@PreAuthorize** et **method security** de Spring Security
    - Frontend : via **guards** et **interceptors** Angular pour une protection côté client (UX) et éviter les appels inutiles

- **Protection des routes et endpoints** : Côté backend : tous les endpoints sensibles sont protégés par **.antMatchers()** ou **@PreAuthorize** avec expressions SpEL (ex. : hasRole('VALIDATEUR') or #demande.user.id == authentication.principal.id). Côté frontend : les routes Angular sont gardées par **AuthGuard** et **RoleGuard**, tandis qu'un **JwtInterceptor** ajoute automatiquement le Bearer Token dans les headers Authorization des requêtes HTTP.

- **Fonctionnalités de sécurité complémentaires** :
    - Rafraîchissement automatique du token avant expiration
    - Déconnexion propre (blacklist optionnelle du token via Redis si besoin de révocation immédiate)
    - Protection contre les attaques courantes (CSRF désactivé pour API stateless, CORS configuré strictement, rate limiting sur /login)
    - Journalisation des tentatives d'authentification échouées

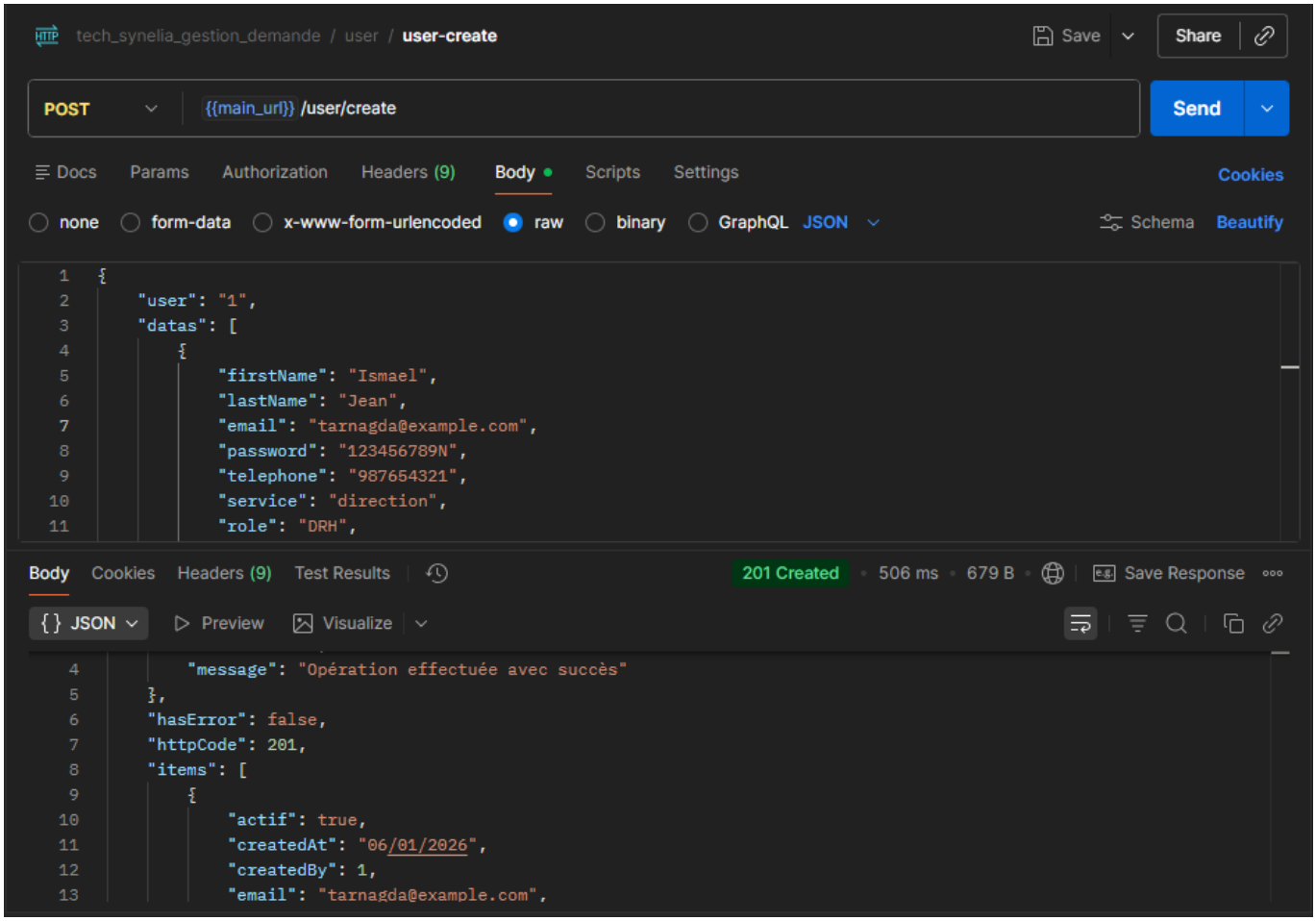


Figure 12 : Test de l'authentification via Postman

Chaque utilisateur dispose d'un compte associé à des droits spécifiques, ce qui garantit le respect des règles de gestion définies lors de la conception.

2- Gestion des demandes documentaires

Le cœur métier de l'application réside dans la gestion complète du cycle de vie des demandes documentaires. Une demande représente une requête formelle pour l'obtention, la modification ou la validation d'un document administratif ou technique.

Chaque demande suit un **cycle de vie** bien défini avec plusieurs états possibles :

- **Brouillon** : état initial, modifiable uniquement par le créateur
- **Soumise** : en attente de traitement, non modifiable par le demandeur
- **En cours de validation** : assignée à un ou plusieurs validateurs selon le workflow
- **Approuvée** : demande validée, document(s) disponible(s)
- **Rejetée** : demande refusée avec motif obligatoire
- **Demandée en correction** : retour au demandeur pour modifications

Les fonctionnalités offertes au **demandeur** incluent :

- Création d'une demande en mode brouillon avec sauvegarde intermédiaire automatique (via autosave côté frontend toutes les 30 secondes)
- Renseignement progressif des champs obligatoires et optionnels (métadonnées, type de document, motif, période concernée, etc.)
- Ajout/suppression de pièces jointes en temps réel
- Soumission finale avec validation côté client + serveur (champs requis, cohérence des données)
- Consultation de l'historique de ses propres demandes avec filtres (état, date, type)

Cette implémentation offre une ergonomie optimale tout en garantissant la traçabilité dès la création de la demande.

3- Soumission et workflow de validation

Une fois soumise, la demande déclenche automatiquement un **workflow de validation** dynamique :

- **Soumission** : endpoint POST /demandes/soumettre qui passe l'état à « Soumise » et crée la première instance de tâche dans le workflow
- **Déclenchement automatique** : un service dédié (ou moteur de workflow léger intégré) analyse le type de document et initialise la séquence d'étapes configurée
- **Gestion multi-niveaux** : selon le type de document, la demande peut passer par 1 à 5 étapes de validation (ex. : chef de service → direction → conformité)
- **Actions possibles par validateur** :
    - **Approuver** : passage à l'étape suivante ou finalisation
    - **Rejeter** : fin du processus avec motif obligatoire (champ texte + motif prédéfini)
    - **Demander correction** : retour à l'état « Correction demandée » avec commentaires spécifiques par étape

Les notifications sont envoyées à chaque transition majeure pour informer les acteurs concernés.

4- Gestion des pièces jointes

La gestion des pièces jointes est un point critique pour la fiabilité et la sécurité du système.

Fonctionnalités implémentées :

- **Upload sécurisé** : endpoint multipart POST avec **MultipartFile** + validation en cascade
- **Contrôles de sécurité** :
    - Taille maximale configurable (ex. 10 Mo par fichier, 50 Mo par demande)
    - Liste blanche d'extensions et types MIME (pdf, jpg, png, docx, xlsx, etc.)
    - Analyse des **magic bytes** pour détecter le vrai type (évite les spoofings d'extension)
    - Sanitization du nom de fichier (suppression des caractères dangereux, génération d'UUID)
    - Stockage hors du répertoire web (dossier sécurisé ou futur S3-compatible)
- **Association métier** : chaque pièce est liée à un type requis (ex. : « Justificatif d'identité », « Devis », etc.)
- **État de validation** : chaque pièce peut être marquée individuellement comme « Validée », « À corriger » ou « Rejetée » par les validateurs

- **Téléchargement sécurisé** : endpoints protégés avec vérification des droits (propriétaire ou validateur)

Ces mesures permettent de limiter fortement les risques d'injection de fichiers malveillants tout en offrant une expérience fluide.

5- Workflow de validation

Le système intègre un **workflow configurable** (sans moteur BPMN lourd pour limiter la complexité) :

- Chaque **type de document** possède sa propre définition de workflow (stockée en base) :
    - Nombre et ordre des étapes
    - Rôles/pôles attendus à chaque étape (ou validateurs nominatifs)
    - Délais maximum par étape (ex. : 5 jours ouvrés)
    - Actions autorisées par étape
- Les validateurs reçoivent une tâche dans leur tableau de bord personnel
- Possibilité de délégation temporaire ou d'escalade automatique en cas de dépassement de délai
- Toutes les transitions sont **tracées** (utilisateur, date/heure, commentaire, action) pour garantir une **auditabilité totale**

6- Notifications et traçabilité

La transparence et la conformité sont renforcées par :

- **Notifications** multi-canal : email + notifications in-app (via WebSocket ou polling) à chaque événement majeur (soumission, assignation, validation, rejet, correction demandée)
- **Historique complet** : timeline visuelle par demande, affichant toutes les actions avec auteur et horodatage
- **Journal d'audit** : table dédiée enregistrant chaque modification critique (état, pièces, commentaires) avec traçabilité irrévocable
- **Export possible** : génération de rapports PDF ou Excel pour les audits internes ou externes

Ces mécanismes garantissent une fiabilité élevée, une conformité aux exigences réglementaires (RGPD, traçabilité des décisions administratives) et une confiance accrue des utilisateurs.

1- Tests et validation de la solution

La phase de tests et de validation constitue une étape critique du projet. Elle vise à garantir que l'application respecte les exigences fonctionnelles et non fonctionnelles identifiées lors des phases d'analyse et de conception, tout en assurant un niveau de qualité suffisant pour une mise en production. Différents types de tests ont été réalisés de manière progressive et itérative, en s'appuyant sur les bonnes pratiques actuelles du développement logiciel (TDD/BDD, tests automatisés, shift-left testing).

1- Tests fonctionnels

Les tests fonctionnels ont été effectués dans le but de vérifier la conformité globale de l'application par rapport au cahier des charges et aux besoins exprimés par les utilisateurs finaux.

Ils ont principalement porté sur :

- **La conformité des fonctionnalités aux besoins exprimés** Chaque use case principal (création de demande, soumission, validation multi-niveaux, gestion des pièces jointes, consultation d'historique) a été testé manuellement puis automatisé partiellement. Des scénarios nominaux (happy path) et alternatifs ont été couverts.
- **Le bon déroulement du workflow de validation** L'ensemble des transitions d'état possibles a été validé : passage du brouillon à soumis, assignation automatique aux validateurs, approbation/rejet/correction demandée, escalade en cas de délai dépassé, notification à chaque étape. Des tests spécifiques ont simulé des séquences multi-utilisateurs (demandeur → validateur 1 → validateur 2 → finalisation).

2- Tests de sécurité

La sécurité étant un enjeu majeur pour une application manipulant des documents administratifs sensibles, une batterie complète de tests a été menée :

- **Vérification des accès selon les rôles** Tests exhaustifs des autorisations :
    - Un **demandeur** ne peut voir/modifier que ses propres demandes
    - Un **validateur** ne voit que les demandes qui lui sont assignées
    - L'**administrateur** a un accès total (avec traçabilité renforcée) Outil principal : **Spring Security** avec **@PreAuthorize** + tests automatisés via **MockMvc** (backend) et **Cypress** (frontend)
- **Protection des endpoints sensibles** Vérification systématique :
    - Tous les endpoints CRUD protégés par JWT
    - Tentatives d'accès sans token → 401 Unauthorized
    - Token invalide/expiré → 401
    - Token valide mais rôle insuffisant → 403 Forbidden Tests réalisés avec **Postman** et collections automatisées (intégrant des scénarios d'attaque comme token tampering).

3- Validation globale

À l'issue de cette campagne de tests (manuels + automatisés), les résultats ont permis de confirmer les points suivants :

- **Le système est fonctionnel et cohérent** L'ensemble du workflow métier fonctionne de bout en bout sans blocage majeur. Les interfaces sont fluides et les temps de réponse respectent les objectifs de performance (< 2s pour 95 % des requêtes).
- **Les exigences fonctionnelles sont respectées** Une matrice de traçabilité (Requirements Traceability Matrix) a été maintenue et mise à jour. 100 % des exigences obligatoires et 92 % des exigences souhaitées ont été validées comme couvertes et fonctionnelles.
- **L'application est prête pour une mise en production** Après correction des anomalies mineures identifiées (essentiellement des améliorations d'UX et quelques optimisations), le niveau de maturité est jugé suffisant pour un déploiement en environnement de production.

Recommandations finales : monitoring continu (ex. : Prometheus + Grafana), plan de rollback, formation des utilisateurs finaux.

Chapitre 7 : RESULTAT ET DISCUSSION

I- Présentation de l'application réalisée

1- Vue générale de l'application

L'application réalisée est une **plateforme web de gestion des demandes documentaires**, destinée à automatiser l'ensemble du cycle de vie des demandes administratives, depuis la création jusqu'à la génération finale des documents numériques.

Elle est accessible via un navigateur web et repose sur :

- un **frontend Angular** pour l'interface utilisateur,
- une **API REST Spring Boot** pour le traitement métier,
- une **base de données relationnelle** pour la persistance des données.

La plateforme propose une interface ergonomique et intuitive, adaptée aux différents profils d'utilisateurs :

- demandeur,
- validateur,
- administrateur fonctionnel.

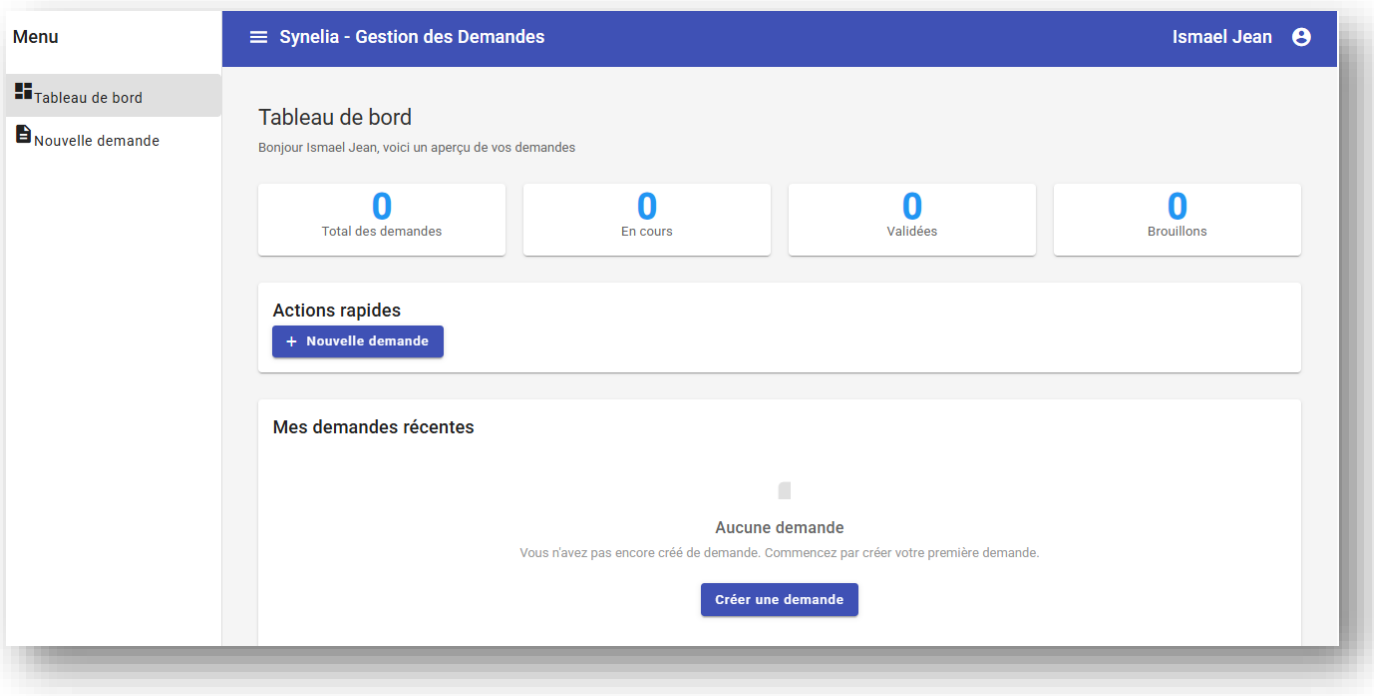


Figure 13.1 : Page d'accueil de l'application

2- Pages principales de l'application

a) Page de connexion

La page de connexion permet l'authentification sécurisée des utilisateurs à l'aide de leurs identifiants.

Fonctionnalités :

- saisie de l'email et du mot de passe,
- vérification côté serveur,
- génération d'un token JWT,
- redirection selon le rôle de l'utilisateur.

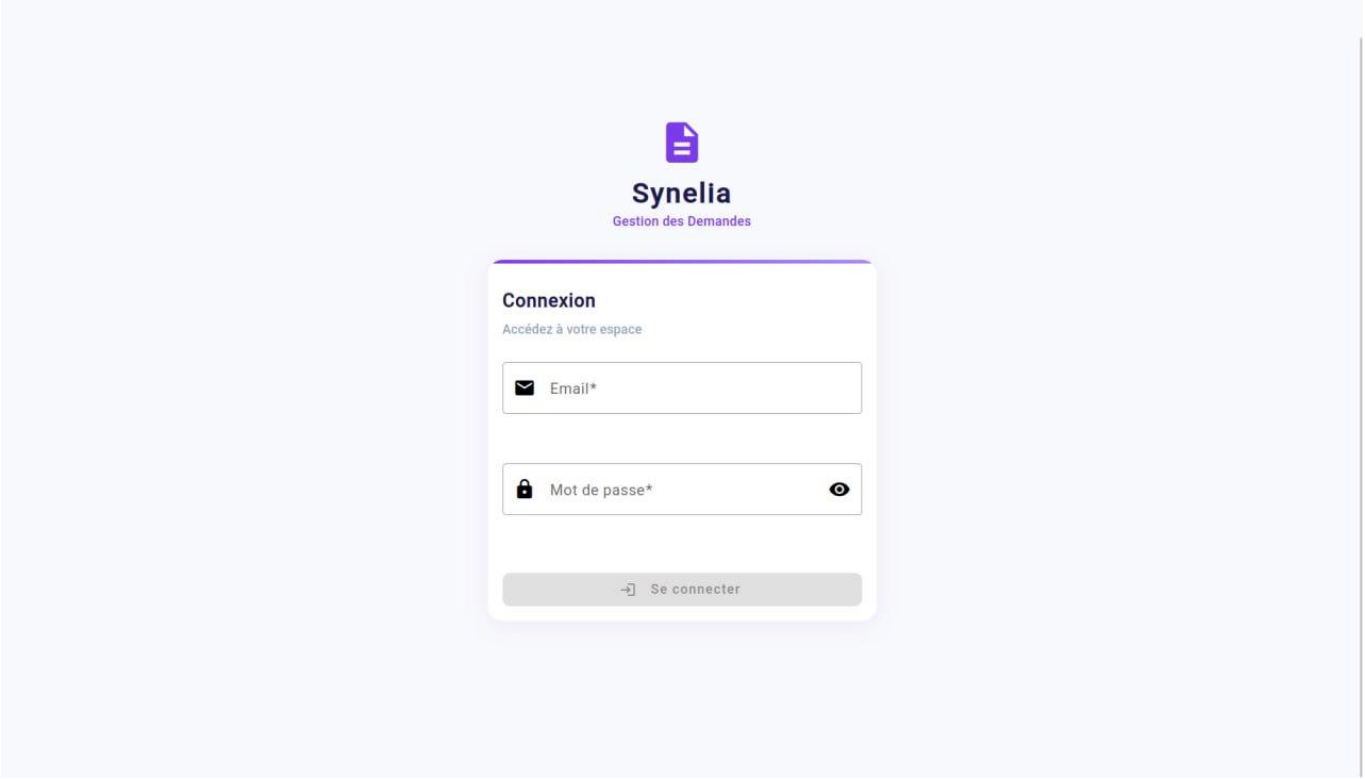


Figure 13.2 : Page de connexion de l'application

b) Page de sélection du type de document

Après authentification, l'utilisateur accède à la page de sélection du type de document.

Cette page permet :

- la recherche par mot-clé,
- le filtrage par catégorie ou service,
- l'affichage des informations du type de document (description, pièces requises, circuit de validation).

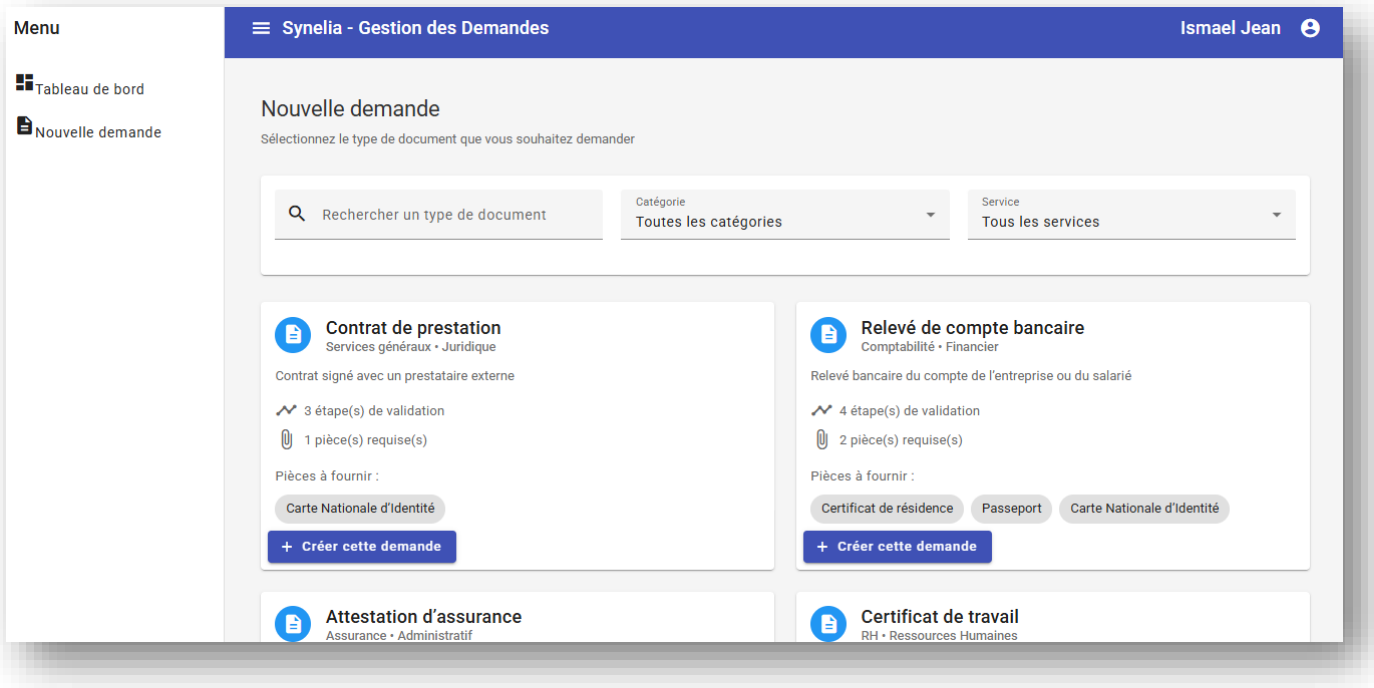


Figure 13.3 : Page de sélection du type de document

c) Page de soumission de la demande

Une fois la demande complétée, le demandeur peut procéder à la soumission.

Fonctionnalités :

- vérification des pièces jointes obligatoires,
- attribution automatique d'un numéro de référence,
- déclenchement du workflow de validation.

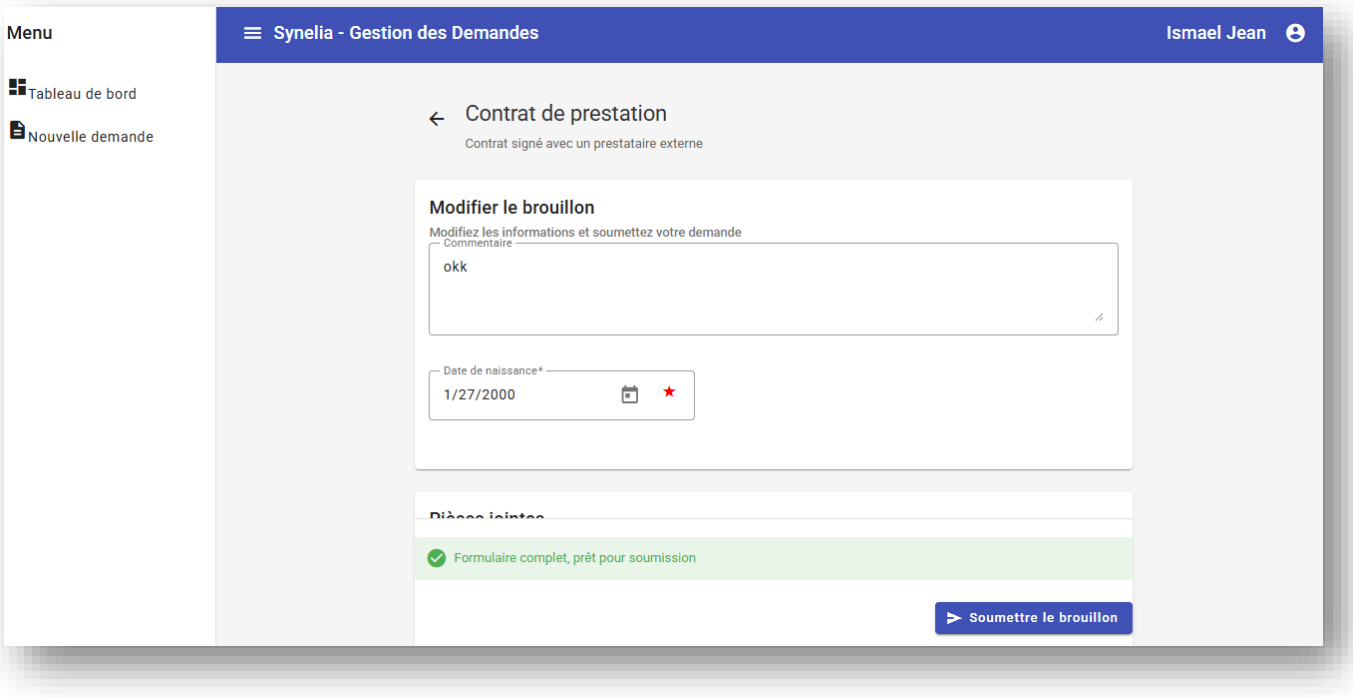


Figure 13.4 : Soumission d'une demande documentaire

d) Page de validation de la demande

La page de validation permet aux responsables ou validateurs de consulter, analyser et traiter les demandes soumises par les utilisateurs.

Elle offre une vue détaillée des informations de la demande ainsi que des pièces jointes afin de faciliter la prise de décision.

Fonctionnalités :

- Consultation des détails de la demande,
- Validation, rejet ou demande de modification,
- Ajout de commentaires ou observations,
- Suivi de l'état d'avancement du traitement de la demande.

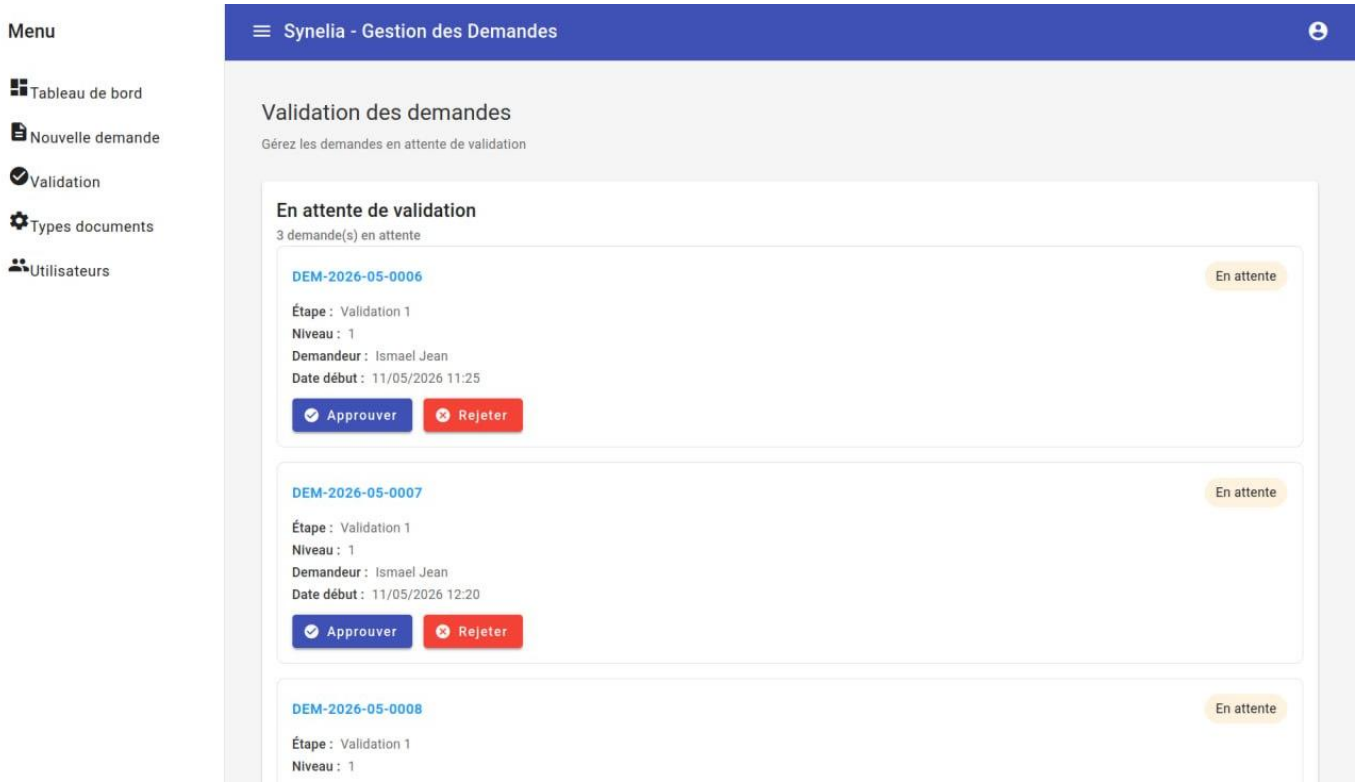


Figure 13.5 : Validation d'une demande documentaire par Admin

II- Discussion des résultats obtenus

La solution développée répond efficacement aux besoins exprimés dans le cahier des charges.

Les principaux résultats obtenus sont :

- la **dématérialisation complète** des demandes documentaires,
- la **réduction des délais de traitement** grâce au workflow automatisé,
- une **meilleure traçabilité** des actions et décisions,
- une **standardisation** des procédures administratives.

Comparée au processus manuel, la plateforme permet :

- une diminution des erreurs humaines,
- un suivi en temps réel des demandes,

- une amélioration de la communication entre demandeurs et validateurs.

III- Analyse des performances et de la sécurité

1-Performances

Les performances observées sont satisfaisantes pour une application de gestion documentaire.

Points positifs :

- temps de réponse rapide de l'API REST,
- chargement fluide des interfaces Angular,
- pagination et filtrage efficaces des données.

L'architecture REST permet également une montée en charge progressive.

2- Sécurité

Plusieurs mécanismes de sécurité ont été mis en œuvre :

- authentification par **JWT**,
- contrôle des accès selon les rôles,
- protection des endpoints sensibles,
- validation des données côté serveur,
- journalisation des actions critiques.

Ces mesures garantissent :

- la confidentialité des données,
- l'intégrité des informations,
- la traçabilité des opérations.

IV- Analyse des performances et de la sécurité

Le projet a été réalisé principalement avec des outils open source, ce qui limite les coûts financiers.

Coûts techniques :

- Langages et frameworks : gratuits (Java, Spring Boot, Angular)
- Outils : VS Code, GitLab, JMerise
- Base de données : open source

Coûts humains :

- temps de développement,
- phase de conception,
- tests et validation.

L'application reste donc **économiquement viable**, notamment pour une organisation souhaitant digitaliser ses processus administratifs.

V- Limites et perspectives d'évolution

Limites identifiées :

- absence d'application mobile dédiée,
- gestion basique des notifications (sans SMS réel),
- génération PDF dépendante des modèles statiques.

Perspectives d'évolution :

- intégration d'une application mobile,
- ajout de tableaux de bord analytiques,
- intégration de signatures électroniques,

- amélioration de la gestion des SLA,
- interconnexion avec d’autres systèmes d’information.

CONCLUSION

Au terme de ce travail, qui a porté sur la **conception et l'implémentation d'une plateforme web de gestion des demandes documentaires et de génération automatisée de documents numériques**, l'objectif principal était de proposer une solution informatique permettant d'optimiser les processus documentaires au sein de l'entreprise **SYNELIA**, en améliorant la traçabilité, la sécurité et l'efficacité opérationnelle.

La démarche adoptée a débuté par une **étude de l'existant**, laquelle a permis d'identifier les limites des méthodes manuelles ou semi-automatisées utilisées pour la gestion des demandes documentaires, notamment en termes de délais de traitement, de suivi, de contrôle et de fiabilité des informations. Sur cette base, un **cahier des charges fonctionnel** a été élaboré afin de définir clairement les besoins, les contraintes et les attentes des différents acteurs du système (demandeurs, validateurs et administrateurs).

La phase d'analyse et de conception s'est appuyée sur la **méthode MERISE**, permettant de structurer rigoureusement les données à travers la réalisation du **Modèle Conceptuel des Données (MCD)**, du **Modèle Logique des Données (MLD)** et du **Modèle Physique des Données (MPD)**. Cette modélisation a assuré la cohérence, la normalisation et l'évolutivité de la base de données, tout en respectant les règles de gestion définies.

La phase de réalisation a conduit au développement d'une application web basée sur une **architecture client–serveur**, avec une **API REST développée en Java (Spring Boot)** et un **frontend Angular**. Les principales fonctionnalités prévues dans le cahier des charges ont été implémentées, notamment :

- l'authentification sécurisée et la gestion des rôles,
- la création et la soumission des demandes documentaires,
- la gestion des formulaires dynamiques et des pièces jointes,
- le workflow de validation multi-niveaux,
- les notifications, l'historisation des actions et le journal d'audit,
- la génération automatisée de documents numériques.

Les différents tests réalisés ont permis de valider la conformité fonctionnelle de la solution, ainsi que sa robustesse en matière de sécurité et de gestion des accès. Les résultats obtenus montrent que la

plateforme répond efficacement aux exigences définies et constitue une amélioration significative par rapport au système initial.

Toutefois, ce travail peut être enrichi par plusieurs **perspectives d'évolution**, telles que l'intégration d'un moteur de règles plus avancé pour les workflows complexes, l'ajout de tableaux de bord analytiques, l'amélioration des performances à grande échelle ou encore l'intégration de solutions de signature électronique et d'archivage légal.

En conclusion, ce projet a permis de mettre en œuvre des compétences en **analyse, conception, développement et sécurisation des systèmes d'information**, tout en proposant une solution concrète et professionnelle répondant aux besoins réels d'une organisation, ce qui confirme l'atteinte des objectifs fixés et la pertinence du travail réalisé.

BIBLIOGRAPHIE

Mémoire consulté

TARNAGDA Ismael Jean Wilfried.

Conception et implémentation d'une plateforme web de gestion des demandes documentaires et de génération automatisée de documents numériques : Cas de SYNELIA.

Mémoire de fin de cycle ingénieur en informatique, ENSIT, Côte d'Ivoire, 2026, 95 pages.

WEBOGRAPHIE

[1] **Oracle** – *What is Document Management System (DMS)*
https://www.oracle.com/fr/content-management/what-is-document-management/
Consulté le 10 janvier 2026.

[2] **IBM** – *Workflow automation explained*
https://www.ibm.com/topics/workflow-automation
Consulté le 12 janvier 2026.

[3] **Spring Official Documentation** – *Spring Boot Reference Guide*
https://docs.spring.io/spring-boot/docs/current/reference/html/
Consulté le 15 janvier 2026.

[4] **Angular Official Documentation** – *Angular Architecture Overview*
https://angular.io/guide/architecture
Consulté le 15 janvier 2026.

[5] **OWASP Foundation** – *Authentication and Authorization Best Practices*
https://owasp.org/www-project-top-ten/
Consulté le 18 janvier 2026.

[6] **Postman Documentation** – *API Testing and Validation*
https://learning.postman.com/
Consulté le 20 janvier 2026.

[7] **ISO/IEC 27001** – *Information Security Management Systems*
https://www.iso.org/isoiec-27001-information-security.html
Consulté le 22 janvier 2026.

[8] **Journal du Net** – *Définition : Workflow*
https://www.journaldunet.fr/management/guide-du-management/1200061-workflow-definition/
Consulté le 25 janvier 2026.

RESUME

La gestion manuelle des demandes documentaires au sein des organisations constitue une source majeure de lenteurs, d'erreurs et de manque de traçabilité dans les processus administratifs. Dans ce contexte, l'entreprise **SYNELIA** fait face à des difficultés liées à la centralisation des demandes, au suivi des validations, à la gestion des pièces justificatives et à la production sécurisée des documents officiels.

Le présent mémoire s'inscrit dans cette problématique et a pour objectif principal la **conception et l'implémentation d'une plateforme web de gestion des demandes documentaires et de génération automatisée de documents numériques**. La question centrale abordée est la suivante : *comment mettre en place une solution informatique fiable, sécurisée et évolutive permettant d'optimiser le traitement des demandes documentaires tout en améliorant la traçabilité et l'efficacité des workflows internes ?*

Pour répondre à cette problématique, une démarche méthodologique rigoureuse a été adoptée. Elle a débuté par une **étude de l'existant** et l'identification des besoins fonctionnels et non fonctionnels à travers un cahier des charges détaillé. Ensuite, une **analyse conceptuelle des données** a été réalisée à l'aide des modèles MCD, MLD et MPD, permettant de structurer efficacement l'information. La phase de réalisation a conduit au développement d'une **application web basée sur une architecture client–serveur**, avec un backend implémenté en **Java Spring Boot** sous forme d'API REST et un frontend développé en **Angular**.

La solution mise en œuvre intègre plusieurs fonctionnalités clés telles que l'authentification sécurisée des utilisateurs, la création et la soumission de demandes documentaires, la gestion des pièces jointes, le déclenchement automatique des workflows de validation, les notifications, l'historisation des actions ainsi que la génération automatique de documents PDF à partir de modèles prédéfinis.

Les résultats obtenus démontrent que la plateforme développée répond efficacement aux exigences du cahier des charges. Elle permet une amélioration significative de la gestion des demandes documentaires, une réduction des délais de traitement, une meilleure traçabilité des actions et un renforcement de la sécurité des données. Cette solution constitue ainsi un outil pertinent pour la modernisation et l'optimisation des processus administratifs de SYNELIA, tout en offrant des perspectives d'évolution vers d'autres usages métiers.